

A Cryptographically Enhanced Web Platform For Controlled and Auditable File Sharing

Submitted in partial fulfilment of the requirements of the degree of

Bachelor of Engineering (B.E.)

in

INFORMATION TECHNOLOGY

by

Hardik Tushar Pawar EU1214036

Harsh Sadanand Nimbare EU1224041

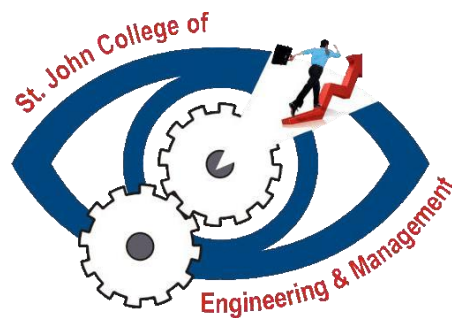
Aryan Gajanan Kasekar EU1224040

Nandini Satyavijay Baisane EU1224022

Under the guidance of

Mrs. Pooja Gharat

Assistant Professor



Department of Information Technology

St. John College of Engineering and Management, Palghar

University of Mumbai

2025–2026

CERTIFICATE

This is to certify that the B.E. project entitled “**A Cryptographically Enhanced Web platform For Controlled and Auditable File Sharing**” is a Bonafide work of **Hardik Tushar Pawar, Harsh Sadanand Nimbare, Aryan Gajanan Kasekar, Nandini Satyavijay Baisane** submitted to University of Mumbai in partial fulfilment of the requirement for the award of the degree of “**Bachelor of Engineering**” in “**Information Technology**” during the academic year **2025–2026**.

Mrs. Pooja Gharat

Project Guide

Dr Arun Saxena

Head of Department

Dr Kamal Shah

Principal

B.E. Project Report Approval

This project report, entitled “A Cryptographically Enhanced Web Platform for Controlled and Auditable File Sharing” by Hardik Tushar Pawar, Harsh Sadanand Nimbare, Aryan Gajanan Kasekar, and Nandini Satyavijay Baisane, is approved for the degree of *Bachelor of Engineering* in *Information Technology* from the *University of Mumbai*.

Examiners

1. _____

2. _____

Date:

Place:

Declaration

We declare that this written submission represents our ideas in our own words, and where others' ideas or words have been included, we have adequately cited and referenced the sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources that have thus not been properly cited or from whom proper permission has not been taken when needed.

1. Hardik Tushar Pawar [EU1214036]

Signature: _____

2. Harsh Sadanand Nimbare [EU1224041]

Signature: _____

3. Aryan Gajanan Kasaker [EU1224040]

Signature: _____

4. Nandini Satyavijay Baisane [EU1224022]

Signature: _____

Date: _____

Place: _____

Abstract

A secure and reliable file-sharing system is essential in today's digital environment, where sensitive data is frequently transmitted over the internet. Traditional file-sharing platforms often lack strong security mechanisms, proper access control, and transparency in user activities, which can lead to data breaches and unauthorized access. This project presents a Cryptographically Enhanced Web Platform for Controlled and Auditable File Sharing, designed to ensure secure data transmission, strict access control, and complete traceability of user actions. The system uses modern cryptographic techniques such as encryption to protect files during storage and sharing. Only authorized users with proper permissions can access or download files, ensuring confidentiality and integrity. Additionally, the platform maintains an audit log system that records all user activities such as file uploads, downloads, and access attempts. This provides accountability and helps in monitoring suspicious behaviour. The system is implemented using a web-based architecture with a user-friendly interface, enabling secure file sharing in real time. The platform also supports role-based access control, allowing administrators to define user permissions effectively. Furthermore, the system ensures data integrity verification, preventing tampering of files during transmission or storage. These features make the system suitable for environments where security and accountability are critical. The proposed system enhances data security, improves trust among users, and provides a scalable solution for secure and auditable file management in various applications such as organisations, educational institutions, and cloud-based services.

Keywords— *Cryptography, Secure File Sharing, Access Control, Audit Logs, Data Security, Web Application, Encryption, Cybersecurity, Role-Based Access Control, Data Integrity, Cloud Security, Secure Communication.*

Table of Contents

	Abstract	v
	List of Figures	viii
	List of Tables	ix
	List of Abbreviations	x
Chapter 1	Introduction	1
	1.1 Motivation	2
	1.2 Problem Statement	2
	1.3 Objectives	3
	1.4 Scope	4
Chapter 2	Review of Literature	5
	2.1 Study of Existing File Sharing Systems	5
	2.2 Cryptography in Secure File Sharing	5
	2.3 Access Control Mechanisms	6
	2.4 Audit Logging and Monitoring Systems	7
	2.5 Limitations of Existing Systems	8
Chapter 3	Requirement Analysis	9
	3.1 Hardware Requirements	9
	3.2 Software Requirements	9
	3.3 Functional Requirements	10
	3.4 Non-Functional Requirements	11
Chapter 4	Design	12
	4.1 DFD Diagram for Secure File Sharing System	12
	4.1.1 Level 0 DFD	12
	4.1.2 Level 1 DFD	13
	4.1.3 Level 2 DFD	14
	4.2 UML Diagrams	15

	4.2.1 Use Case Diagram for Secure File Sharing System	15
	4.2.2 Class Diagram for Secure File Sharing System	16
	4.2.3 Activity Diagram for Secure File Sharing System	17
	4.2.4 Sequence Diagram for Secure File Sharing System	18
	4.2.5 Timeline/Gantt Chart for Secure File Sharing System	19
	4.2.6 Work Breakdown Structure (WBS) Chart for Secure File Sharing System	19
Chapter 5	Report on Present Investigation	20
	5.1 Methodology/Proposed System	21
	5.1.1 Architecture diagram of the System	21
	5.2 Implementation	21
	5.2.1 Algorithm/Flowchart	22
	5.2.2 System Data / Log Data	23
	5.2.3 GUI	24
Chapter 6	Results and Discussion	26
Chapter 7	Conclusion	31
	References	33
Appendix	Technologies Used	35
	Publication	38
	Acknowledgments	39

List of Figures

Figure No.	Figure Name	Page No.
4.1.1	Level 0 DFD for Secure File Sharing System	8
4.1.2	Level 1 DFD for Secure File Sharing System	8
4.1.3	Level 2 DFD for Secure File Sharing System	9
4.1	Use Case Diagram for Secure File Sharing System	9
4.2	Class Diagram for Secure File Sharing System	10
4.3	Activity Diagram for Secure File Sharing System	11
4.4	Sequence Diagram for Secure File Sharing System	12
4.5	Timeline/Gantt Chart for Secure File Sharing System	12
4.6	Work Breakdown Structure (WBS) Chart for Secure File Sharing System	13
5.1	Architecture /Block diagram of the system	14
5.2	GUI for User Login and File Upload	18
5.3	GUI for Secure File Sharing and Access Control	18
5.4	GUI for Audit Logs and Activity Monitoring	19

List of Tables

Table No.	Table Name	Page No.
3.1	System Data	18
3.2	Audit Log Data	18

List of Abbreviations

AES	Advanced Encryption Standard
RBAC	Role-Based Access Control
WBS	Work Breakdown System
DFD	Data Flow Diagram
GUI	Graphical User Interface
API	Application Programming Interface

CHAPTER 1

INTRODUCTION

Effective and secure data sharing is a fundamental requirement in today's digital world, where individuals and organisations frequently exchange sensitive information over the internet. Traditional file sharing methods, such as email attachments and cloud storage platforms, often lack strong security mechanisms, proper access control, and transparency in user activities [1]. This makes them vulnerable to data breaches, unauthorized access, and misuse of confidential information [2]. With the increasing reliance on digital communication, ensuring data confidentiality, integrity, and accountability has become a critical challenge [3]. Various file sharing systems exist today, but many of them do not provide end-to-end security or detailed monitoring of user actions [4]. In many cases, once a file is shared, there is limited control over how it is accessed or distributed further. Additionally, the absence of proper audit mechanisms makes it difficult to track who accessed or modified the data, leading to potential security risks [5]. These limitations highlight the need for a more secure and controlled approach to file sharing.

In real-world scenarios, organisations often deal with highly sensitive data such as financial records, personal information, and confidential business documents, which require strict security measures. Any compromise in such data can lead to serious consequences including financial loss, reputational damage, and legal issues. Therefore, it becomes essential to implement systems that not only protect data but also ensure controlled accessibility. Furthermore, with the rapid growth of cloud-based services, users are increasingly dependent on online platforms, making security a top priority. The lack of user awareness and weak authentication practices also contribute to vulnerabilities in existing systems. Hence, there is a strong need to design a solution that addresses these challenges effectively while maintaining ease of use.

Advancements in cryptography, web technologies, and cloud computing offer effective solutions to these challenges [6]. Encryption techniques such as symmetric and asymmetric cryptography can be used to protect data during storage and transmission, ensuring that only authorized users can access the files [7]. Role-Based Access Control (RBAC) mechanisms further enhance security by restricting access based on user roles and permissions [8]. Moreover, audit logging systems can record user activities, providing transparency and accountability in the system. This project proposes a Cryptographically Enhanced Web Platform for Controlled and Auditable File Sharing, which integrates secure encryption techniques with access control and activity monitoring. The system allows users to upload, share, and download files securely while ensuring that all actions are logged and traceable. The platform is designed as a web-based application with a user-friendly interface,

enabling real-time, secure communication and file management. The primary goal is to develop a system that ensures data security, controlled access, and complete auditability, making it suitable for applications where confidentiality and trust are essential.

1.1 Motivation

In the modern digital era, the exchange of files over the internet has become a common practice in both personal and professional environments. However, most existing file-sharing platforms focus primarily on convenience rather than security. Sensitive data such as personal documents, financial records, and confidential organisational files are often shared without proper encryption or access control, increasing the risk of data breaches and unauthorized usage. In many cases, users are unaware of the potential risks involved in sharing data through unsecured platforms, which further increases vulnerability. As the volume of digital data continues to grow, the need for secure and efficient file-sharing mechanisms becomes even more important.

Another major concern is the lack of transparency and accountability in traditional systems. Once a file is shared, there is limited control over who accesses it, how it is used, or whether it is further distributed. This absence of monitoring mechanisms creates challenges in maintaining trust and ensuring data privacy. Additionally, cyber threats such as hacking, data leakage, and unauthorized access are increasing rapidly, making secure file sharing a critical requirement. The inability to track user activities also makes it difficult to identify malicious actions or security breaches in time. This highlights the importance of implementing systems that provide complete visibility of user interactions with shared data.

The motivation behind this project is to develop a system that ensures secure, controlled, and auditable file sharing using cryptographic techniques. By integrating encryption, role-based access control, and audit logging, the system aims to provide a reliable platform where users can share files with confidence. The use of modern web technologies further enhances system performance and usability, making it accessible to a wide range of users. The goal is to enhance data security, improve user trust, and provide a solution that addresses the limitations of existing file-sharing systems. Such a system can be highly beneficial in environments where data confidentiality and controlled access are of utmost importance.

1.2 Problem Statement

In today's digital environment, the rapid exchange of files over the internet has introduced significant security challenges. Many existing file-sharing systems lack proper encryption, access control, and monitoring mechanisms, making them vulnerable to unauthorized access, data leakage, and misuse of sensitive information. Once a file is shared, there is often no control over how it is accessed or distributed further. This creates serious concerns for individuals and organisations that rely on secure

data communication for their daily operations. In addition, the absence of proper security measures increases the risk of cyberattacks, which can compromise sensitive information and lead to financial and reputational losses. The growing dependency on digital platforms has further amplified the need for reliable and secure file-sharing solutions.

This project aims to address these issues by developing a secure file-sharing system that uses cryptographic techniques to protect data, implements role-based access control to restrict unauthorized usage, and maintains audit logs to track user activities. The system focuses on ensuring data confidentiality, integrity, and accountability during file sharing. By combining these security mechanisms, the proposed system provides a more controlled and transparent environment for file exchange. It also enables users to monitor and verify all actions performed within the system, thereby improving trust and reliability. The solution is designed to be efficient, scalable, and suitable for real-world applications where data security is a primary concern.

Furthermore, the system aims to reduce the chances of data misuse by enforcing strict access policies and continuous monitoring. It also helps in identifying suspicious activities through detailed logging, allowing timely action against potential threats. The platform is designed to support secure communication channels, ensuring that data remains protected throughout the entire process. Additionally, it enhances user confidence by providing a reliable and transparent file-sharing mechanism. The system can also be extended to include advanced security features in the future, making it adaptable to evolving cybersecurity challenges. Overall, the project contributes towards building a safer digital environment for secure information exchange.

1.3 Objectives

The objectives of this project focus on developing a secure and efficient file-sharing system that ensures data protection and controlled access. The primary objective is to develop a secure web-based file sharing system using cryptographic techniques to safeguard sensitive information. The system aims to implement strong encryption and decryption mechanisms to protect files during both storage and transmission, ensuring that data remains confidential at all times. Another important objective is to provide role-based access control, which restricts unauthorized access and allows only permitted users to view or modify files. Additionally, the system is designed to maintain audit logs that track user activities such as file uploads, downloads, and access attempts, thereby improving transparency and accountability.

Furthermore, the project aims to design a user-friendly interface that enables users to manage files efficiently without compromising security. It also focuses on ensuring data integrity, so that files are not altered or tampered with during transfer or storage. The system is intended to enhance reliability

by providing secure and consistent performance even with multiple users. Another objective is to create a scalable solution that can be adapted for larger systems in the future. Overall, the project seeks to combine security, usability, and performance to deliver a robust file-sharing platform suitable for real-world applications.

In addition, the project aims to minimize the risks associated with data breaches by implementing multiple layers of security within the system. It also focuses on improving user trust by ensuring that all file operations are secure and traceable. The system is designed to provide quick and efficient file access while maintaining strict security protocols. Another objective is to ensure compatibility with modern web standards so that the platform can be accessed across different devices and environments. The project also considers future enhancements such as integrating advanced authentication mechanisms and improving system performance. Overall, these objectives contribute to building a comprehensive and dependable secure file-sharing solution.

1.4 Scope

The scope of this project is to develop a secure and reliable web-based file sharing system that ensures data protection and controlled access. The system focuses on implementing cryptographic techniques to secure files during storage and transmission, along with role-based access control to manage user permissions. It also includes an audit logging mechanism to monitor and record all user activities, providing transparency and accountability. The system can be used in organisations, educational institutions, and other environments where secure file sharing is required. The platform is designed to be scalable and adaptable for future security enhancements. Additionally, the system supports secure communication between users and ensures that only authorized individuals can access shared files. It also helps in preventing data tampering and unauthorized modifications. Future enhancements may include integration with cloud storage, multi-factor authentication, and improved performance for handling large-scale data.

The system is designed to balance security and usability, ensuring users can interact with it easily without unnecessary complexity. It supports multiple users at the same time while maintaining reliable performance and strong data protection. The solution can be extended to handle different file formats and large file sizes, making it suitable for various applications. It can also integrate with existing organisational systems to improve security practices. Built with modern web technologies, the platform remains accessible across multiple devices and browsers. The scope also includes advanced security features like intrusion detection, automated alerts, and customization based on user or organisational needs. Overall, it aims to provide a flexible, secure, and future-ready file-sharing solution.

CHAPTER 2

REVIEW OF LITERATURE

2.1 Study of Existing File Sharing Systems

Various file-sharing systems have been developed over the years to enable users to store, access, and transfer files over the internet. These systems have significantly improved communication and collaboration, especially in organisations and educational environments. Popular file-sharing platforms mainly focus on providing ease of use, quick access, and efficient storage solutions. However, many of these systems prioritize usability over security, which creates several vulnerabilities.

Vurukonda and Rao highlighted that data storage in cloud environments faces critical security challenges including insider attacks, data loss, and service availability failures. They emphasized that without proper security mechanisms, cloud-stored data remains highly vulnerable to unauthorized access and exploitation [1]. Rawal and Vivek demonstrated that while cloud-based file sharing offers scalability and convenience, secure access and data confidentiality remain significant open problems. Their work proposed a secure cloud storage and file sharing model that addresses authentication and encryption gaps found in traditional platforms [2]. Khashan proposed a user-side encrypted file system for secure outsourcing and sharing of cloud data, addressing the gap where cloud providers cannot be fully trusted as security authorities. This approach ensures that data is encrypted before it leaves the user's device, providing end-to-end protection [3].

Pitchai et al. proposed searchable encrypted data file sharing using public cloud services, demonstrating that encryption and keyword searchability can coexist effectively without compromising data confidentiality. Their method allows users to search encrypted files without revealing the actual content to the server [4]. Ali et al. introduced SeDaSC, a secure data sharing scheme in clouds that supports data confidentiality and forward and backward access control when group membership changes. This scheme is particularly useful in dynamic environments where users frequently join or leave a shared group [5].

The adoption of cloud-based file sharing has improved accessibility and convenience, allowing users to access files from anywhere at any time. However, this convenience also introduces security risks, especially when proper configurations are not implemented. Many platforms rely on third-party services for storage, which can introduce additional vulnerabilities. The absence of real-time alerts for suspicious activities further weakens existing systems, as users are often unaware of unauthorized access until significant damage has already occurred. These challenges highlight the

need for a more secure and controlled file-sharing system that ensures both usability and strong security features.

2.2. Cryptography in Secure File Sharing

Cryptography plays a fundamental role in securing data during both storage and transmission in modern file-sharing systems. It ensures that sensitive information remains protected from unauthorized access and interception. Researchers have explored various encryption techniques including symmetric encryption, asymmetric encryption, and hybrid approaches to enhance data security in cloud environments.

Xu et al. proposed a secure fine-grained access control and data sharing scheme for dynamic groups in the cloud using advanced cryptographic techniques. Their scheme enables precise permission management even as group membership changes over time, ensuring that only currently authorized users can access shared data [6]. Li et al. proposed a lightweight secure data sharing scheme for mobile cloud computing that incorporates efficient cryptographic access control suitable for resource-constrained devices, demonstrating that strong security does not have to come at the cost of system performance [7]. Li et al. developed TMACS, a robust and verifiable threshold multi-authority access control system that distributes cryptographic trust among multiple authorities to prevent single points of failure and improve overall system reliability [8].

Kao et al. addressed the critical issue of key management through uCloud, a user-centric key management scheme for cloud data protection that places full key control in the hands of the data owner rather than the cloud service provider. This approach significantly reduces the risk of unauthorized decryption by untrusted third parties [9]. Viswanath and Krishna proposed a hybrid encryption framework for securing big data storage in a multi-cloud environment, combining AES-based symmetric encryption with asymmetric techniques to achieve both computational efficiency and a high level of security [10].

Modern file-sharing systems increasingly rely on hybrid encryption techniques that combine the speed of symmetric encryption with the security of asymmetric encryption. Hashing algorithms are also used alongside encryption to ensure data integrity by generating unique hash values for files. Any modification to a file results in a change in its hash value, making tampering easy to detect. Continuous advancements in cryptographic research aim to address evolving cyber threats and improve the overall robustness of secure file-sharing systems.

2.3 Access Control Mechanisms

Access control is a key component of any secure file-sharing system, as it ensures that only authorized users can access specific resources. Without proper access control, even strongly

encrypted data can be compromised if permissions are incorrectly assigned or managed. Various access control models have been proposed and studied to address these challenges in cloud-based environments.

Verma et al. proposed a novel AES-DES-RSA hybrid cryptographic approach for cloud data storage, showing that layered encryption combined with proper access control significantly reduces the risk of data compromise and unauthorized access [11]. Mahalle and Shahade demonstrated that implementing a hybrid RSA and AES encryption algorithm in cloud systems effectively enhances data security while maintaining acceptable performance overhead, making it a practical solution for real-world deployments [12]. Negi et al. proposed a hybrid cryptographic approach for secure cloud-based file storage that combines multiple encryption algorithms with structured access control to improve both security and system performance [13].

Qin et al. surveyed proxy re-encryption techniques for secure data sharing in cloud computing and highlighted how re-encryption schemes can enforce fine-grained access delegation without exposing private keys to the cloud server or unauthorized parties. This technique is particularly useful when access rights need to be transferred between users securely [14]. Gupta et al. conducted a comprehensive systematic review of secure data storage and sharing techniques for cloud environments, confirming that integrating access control with encryption and real-time monitoring is essential for achieving comprehensive and reliable data protection [15].

Role-Based Access Control (RBAC) remains one of the most widely used models, where permissions are assigned based on user roles such as administrator, editor, or viewer. However, modern systems are increasingly exploring attribute-based and dynamic access control models that consider additional contextual factors such as user behavior, location, and time of access. Strong authentication methods such as multi-factor authentication (MFA) are also being integrated with access control systems to add an additional layer of identity verification and reduce the risk of unauthorized privilege escalation.

2.4 Audit Logging and Monitoring Systems

Audit logging and monitoring systems are essential for maintaining transparency, accountability, and security awareness in file-sharing platforms. These systems record all user activities including file uploads, downloads, modifications, deletions, and access attempts, creating a detailed trail of every action performed within the system.

Yang et al. surveyed data security and privacy protection mechanisms for cloud storage and identified the absence of real-time anomaly detection and comprehensive activity logging as key weaknesses in many existing systems. Their work emphasized that without proper logging frameworks, detecting and responding to security incidents becomes significantly more difficult

[16]. Detailed logs allow organisations to track user behaviour, detect unauthorized access patterns, and conduct thorough forensic analysis after a security incident has occurred. This information is crucial for taking corrective actions, improving system defenses, and preventing future breaches.

Without real-time alerts, administrators may not respond quickly enough to potential threats, allowing attackers to cause significant damage before being detected. Advanced monitoring systems address this issue by providing automated alerts for unusual activities such as multiple failed login attempts, access from unrecognized locations, or unauthorized file access attempts. These real-time alerts enable administrators to take immediate action and minimize potential damage to the system and its data.

Proper encryption of stored audit logs and restricted access to the logging system are also important considerations. If log data is tampered with or deleted, the reliability and usefulness of the entire audit system is compromised. Modern platforms include visualization tools and dashboards that simplify log analysis and provide clear insights into overall system activity patterns. Continuous monitoring helps organisations identify recurring attack patterns, detect insider threats at an early stage, and maintain compliance with data security standards and regulations. Regular review of logs ensures that no suspicious activity goes unnoticed and that the system remains in a secure and well-monitored state.

2.5 Limitations of Existing Systems

Despite significant advancements in file-sharing technologies, several limitations continue to affect the security and reliability of current systems. Understanding these limitations is important for designing a more robust and integrated solution.

Many existing systems implement encryption, access control, and audit logging as separate and loosely connected components rather than as a unified security framework. This fragmented approach leads to inconsistencies in security policies, configuration errors, and gaps that attackers can exploit. Scalability remains a persistent concern, as system performance often degrades noticeably with increasing data volumes and growing numbers of concurrent users. Security patches and updates are not regularly applied in many platforms, leaving them exposed to newly discovered vulnerabilities and evolving attack techniques.

CHAPTER 3

REQUIREMENTS ANALYSIS

3.1 Hardware Requirements

Sr. No.	Component	Specification
1	Processor (PC/Laptop)	Intel Core i5 or above
2	RAM	8 GB (7.87 GB usable)
3	Storage	Minimum 256 GB SSD / HDD, Maximum 1TB
4	Operating System	Windows 10/11

3.2 Software Requirements

Sr. No.	Component	Specification
1	Programming Language (Frontend)	HTML + Python (Django)
2	Programming Language (Backend)	Python (Django)
3	Frontend Framework	HTML + Django
5	Backend Framework	Django
6	Database	Supabase
7	Authentication	Django Authentication System, Django-Allauth, Google OAuth, PBKDF2 Password Hashing.
10	Image Storage / CDN	Supabase
13	IDE / Code Editor	VS Code
14	Version Control	Git & GitHub
15	Deployment Platform	Vercel

3. 3 Functional Requirements

1. User Registration and Login Functionality

The system provides secure user registration and login features for authorized access. Users can create accounts using basic details and log in with valid credentials. Passwords are stored in encrypted form for security. Authentication ensures only verified users can access the platform.

2. Secure File Upload and Download System

Users can upload files securely through the web platform and download them when needed. Uploaded files are stored safely on the server with proper permissions. Only authorized users can access shared files. This ensures secure and controlled file transfer between users.

3. Encryption and Decryption of Files

The system encrypts files before storing them to protect sensitive information. Encrypted files remain unreadable to unauthorized users. When an authorized user downloads a file, the system decrypts it securely. This maintains confidentiality during storage and sharing.

4. Role-Based Access Control for Users

The platform uses Role-Based Access Control (RBAC) to manage user permissions. Different roles such as admin and normal user are assigned specific privileges. Admins can manage users and files, while normal users have limited access. This prevents unauthorized operations on the system.

5. Maintain Audit Logs of User Activities

The system records user activities such as login attempts, file uploads, downloads, and access requests. These logs help track system usage and suspicious behavior. Administrators can review logs for monitoring and security purposes. This improves accountability and transparency.

3. 4 Non-Functional Requirements

1. High Security

The system must ensure strong security by protecting files, user data, and login credentials from unauthorized access. Encryption and secure authentication methods are used to maintain confidentiality.

2. Reliability

The platform should work consistently without failures during file upload, download, or user access. It must provide stable performance even when multiple users use the system simultaneously.

3. High Performance

The system should process user requests quickly and efficiently. File operations such as upload, encryption, and download must be completed with minimum delay.

4. Fast Response Time

Users should receive quick responses while navigating pages or performing actions. This improves usability and provides a smooth experience.

5. User-Friendly Interface

The platform should have a simple and easy-to-use interface so users can perform tasks without technical difficulty. Clear navigation improves productivity.

6. Scalability

The system should support future enhancements such as more users, larger storage, and additional security features without affecting performance.

CHAPTER 4

DESIGN

4.1 Dataflow Diagrams (DFDs)

4.1.1 Level 0 DFD

This Level 0 DFD represents the overall working of the Cryptex secure file sharing system. It shows how external entities such as Registered User, Recipient, Administrator, and Supabase cloud storage interact with the central system. Users can upload and share encrypted files, recipients can request and download authorized files, while the administrator manages users and monitors operations. The diagram provides a high-level view of data flow, secure storage, and controlled access within the platform.

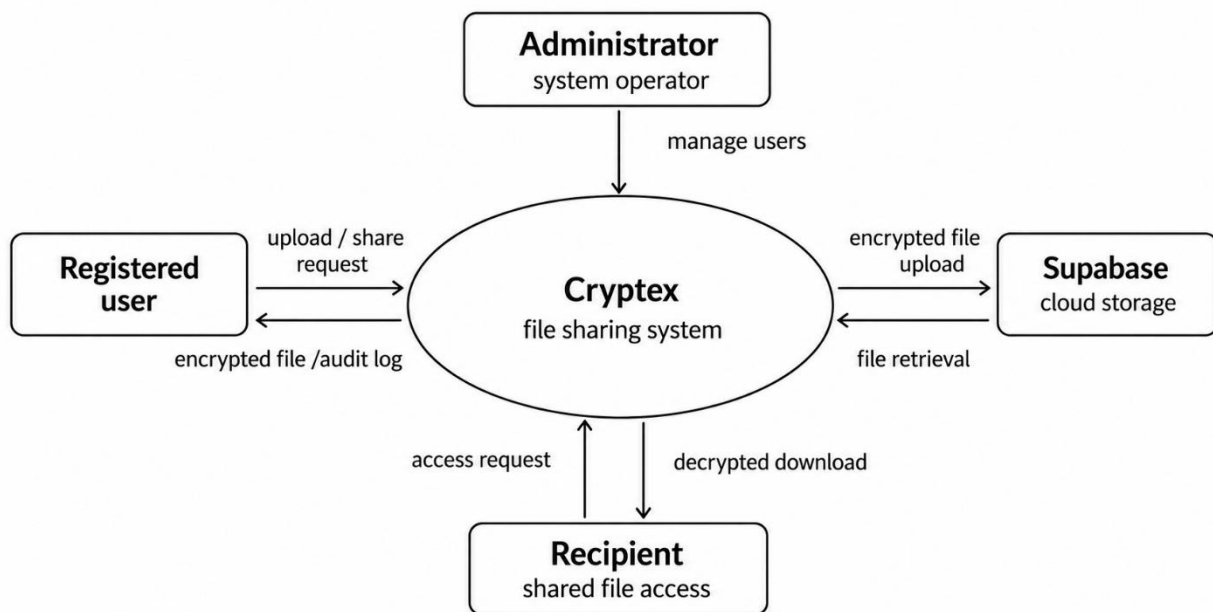


Figure 4.1: Level 0 DFD diagram for Secure File Sharing System

4.1.2 Level 1 DFD

This Level 1 DFD illustrates the internal functional modules of the Cryptex secure file sharing system in detail. It includes processes such as user authentication, key generation, file encryption and upload, secure sharing, controlled download with decryption, audit logging, and admin monitoring. The diagram also shows interaction with data stores like user keys, shared file metadata, audit logs, and Supabase storage. It explains how security mechanisms and data flow are managed step-by-step within the system.

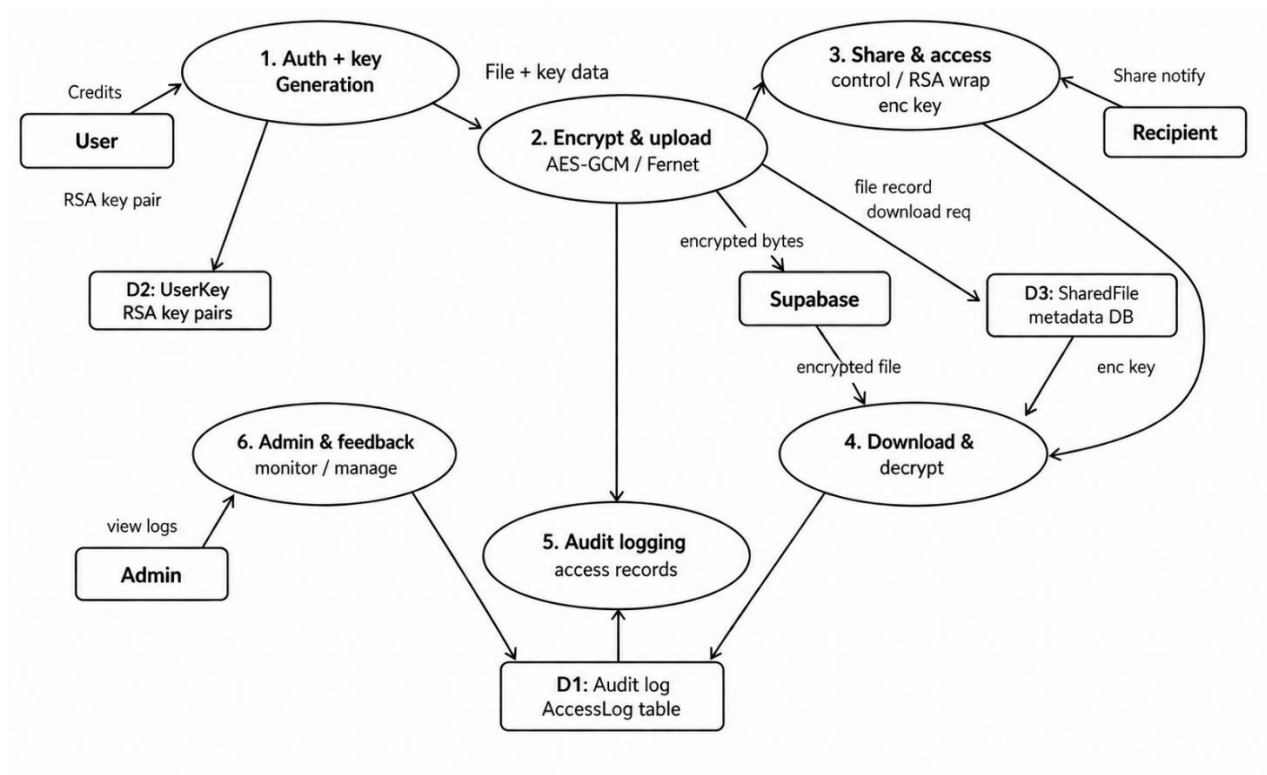


Figure 4.2: Level 1 DFD diagram for Secure File Sharing System

4.1.3 Level 2 DFD

This Level 2 DFD represents the detailed internal workflow of the Cryptex secure file sharing system. It illustrates processes such as user registration and authentication, RSA key generation, file encryption and upload, secure key sharing, recipient-side file decryption, and audit logging. The diagram also shows interaction with data stores including PostgreSQL for user credentials, Supabase storage for encrypted files, and AccessLog records for monitoring activities. It provides a complete view of how security, controlled access, and traceability are maintained within the platform.

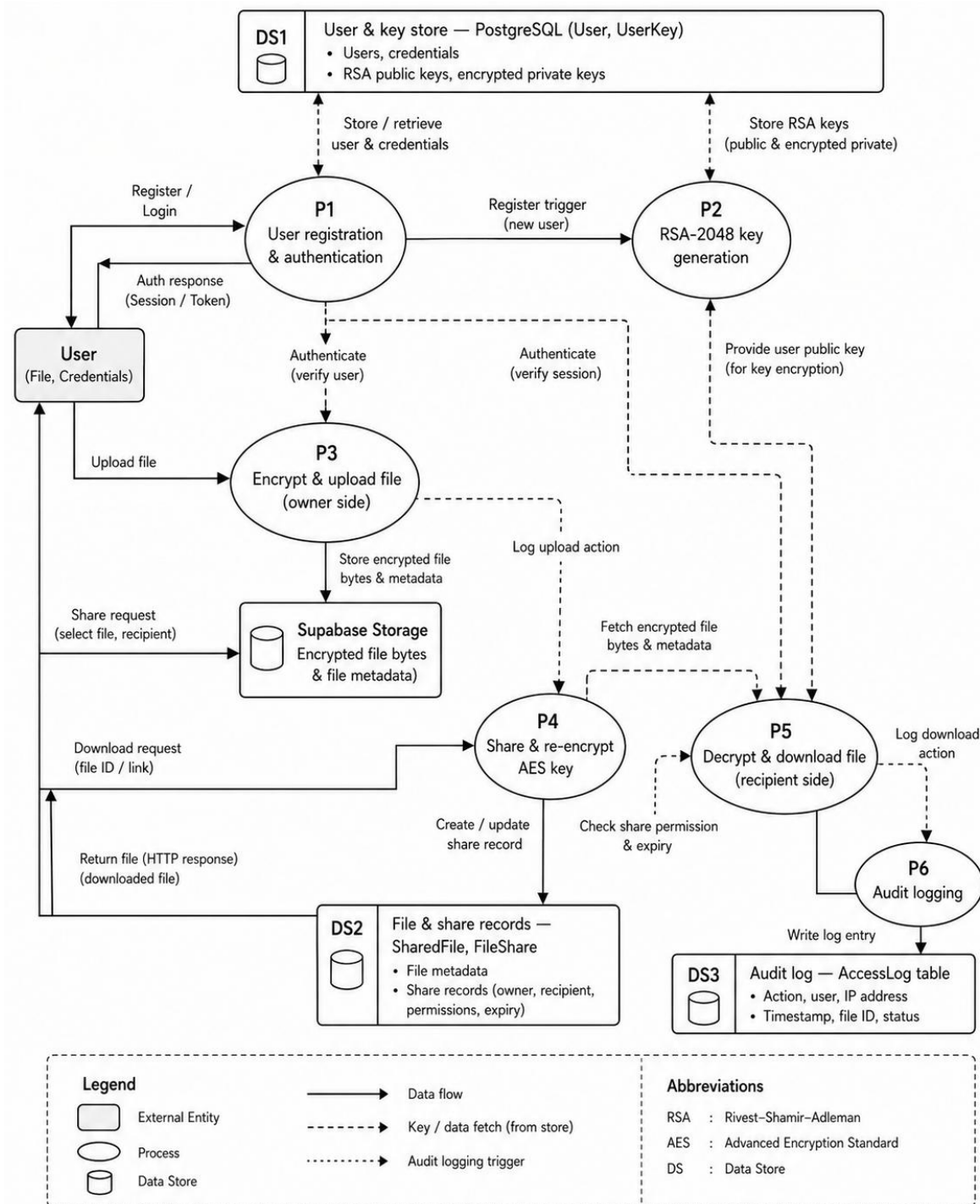


Figure 4.3: Level 2 DFD Diagram for Secure File Sharing System

4.2 UML Diagrams

4.2.1 Use Case Diagram

This Use Case Diagram represents the interaction between User and Admin with the Cryptex secure file sharing system. It shows major functionalities such as registration/login, file upload, download, file viewing, audit log monitoring, user management, and feedback handling. Normal users can access file operations and submit feedback, while the admin can manage users and review feedback logs. The diagram highlights role-based access and the core services provided by the system.

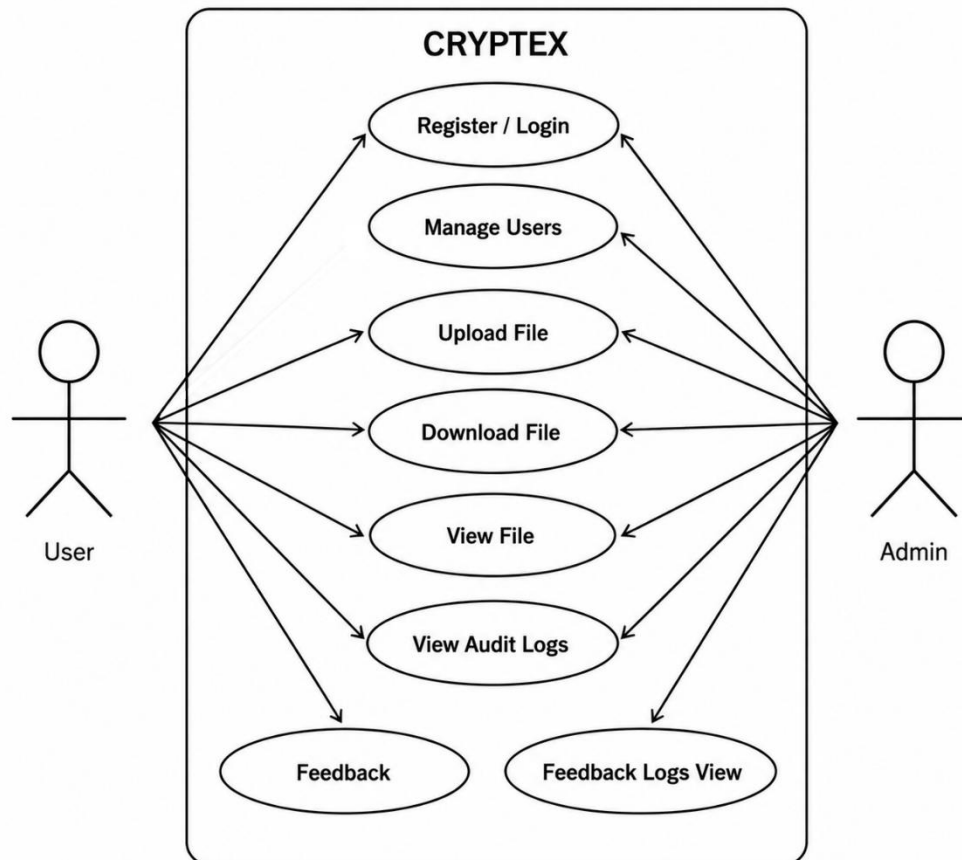


Figure 4.4: Use case diagram for Secure File Sharing System

4.1.3 Class Diagram

This Class Diagram represents the database structure and object relationships of the Cryptex secure file sharing system. It includes main classes such as User, UserKey, SharedFile, FileShare, Feedback, and AccessLog with their attributes and methods. The diagram shows how users own files, share encrypted files with others, generate keys, submit feedback, and create access logs for monitoring. It provides a clear view of system architecture and data relationships used for secure file management.

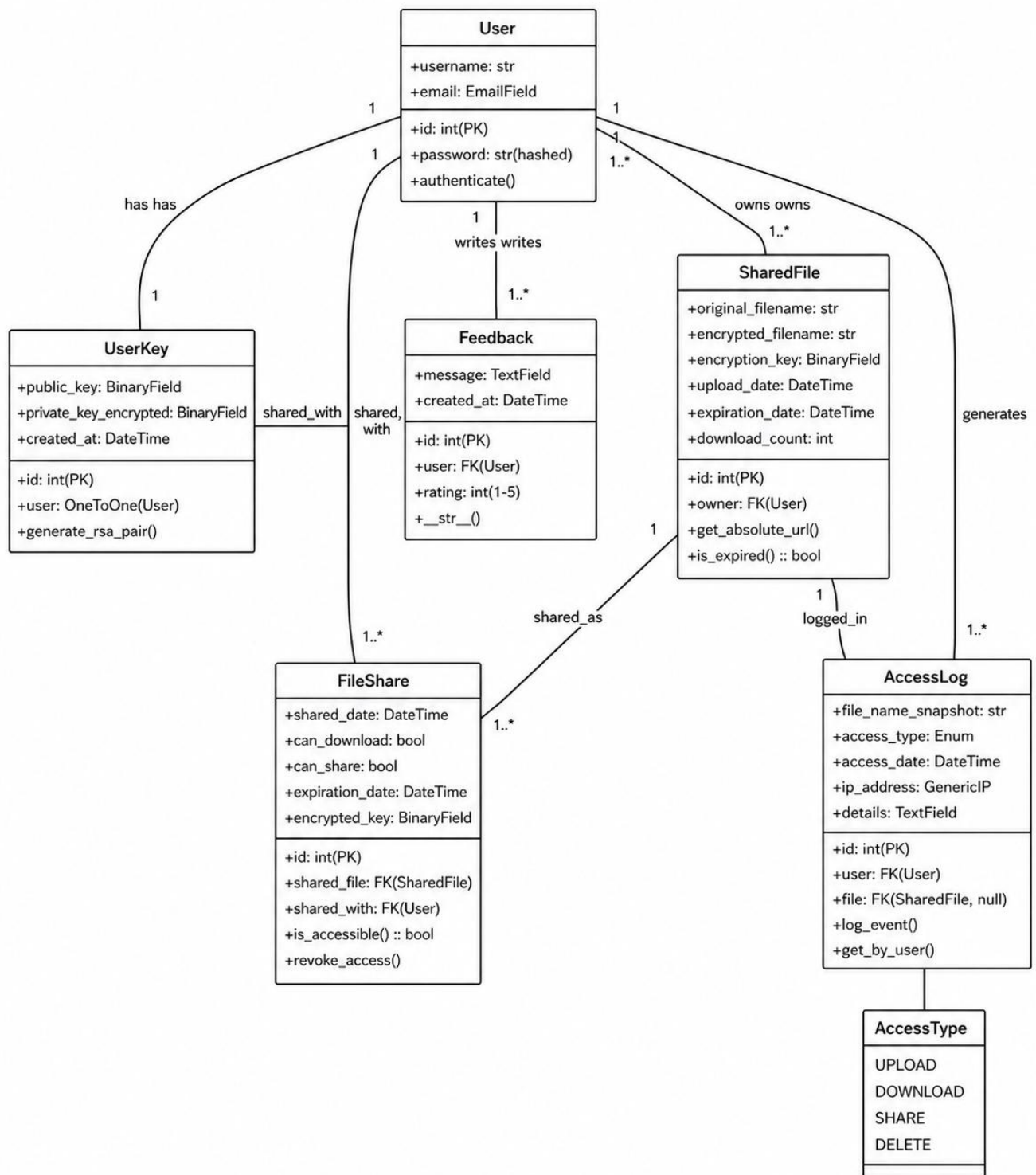


Figure 4.5: Class diagram for Secure File Sharing System

4.2.3 Activity Diagram

This Activity Diagram represents the complete workflow of the Cryptex secure file sharing platform from user login to file access. It shows interactions between the User/Browser, Django Backend, and Supabase database during registration, encryption, file upload, sharing, permission verification, download, and decryption processes. Decision points such as new user validation, access permission, and file expiry are included to ensure secure operations. The diagram clearly explains the sequential activities, data flow, and audit logging performed within the system.

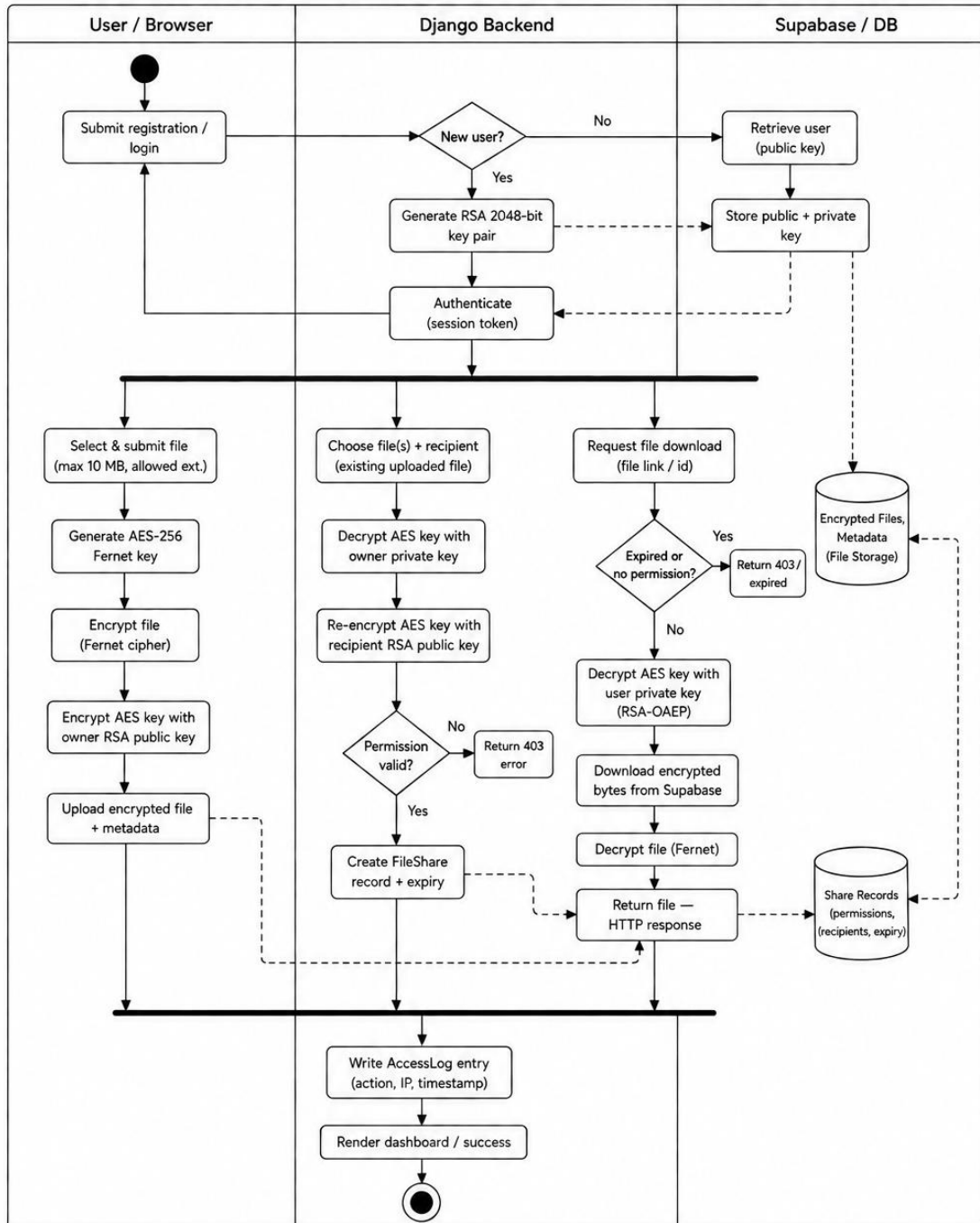


Figure 4.6: Activity diagram for Secure File Sharing System

4.2.4 Sequence Diagram

This Sequence Diagram illustrates the step-by-step interaction between system components during secure file upload and download operations in Cryptex. It shows communication among Browser, Django Views, Encryption Utilities, Django ORM, and Supabase storage. During upload, files are validated, encrypted, stored securely, and logged, while during download, permissions are checked, files are retrieved, decrypted, and delivered to the user. The diagram explains the chronological flow of requests and responses for secure file handling.

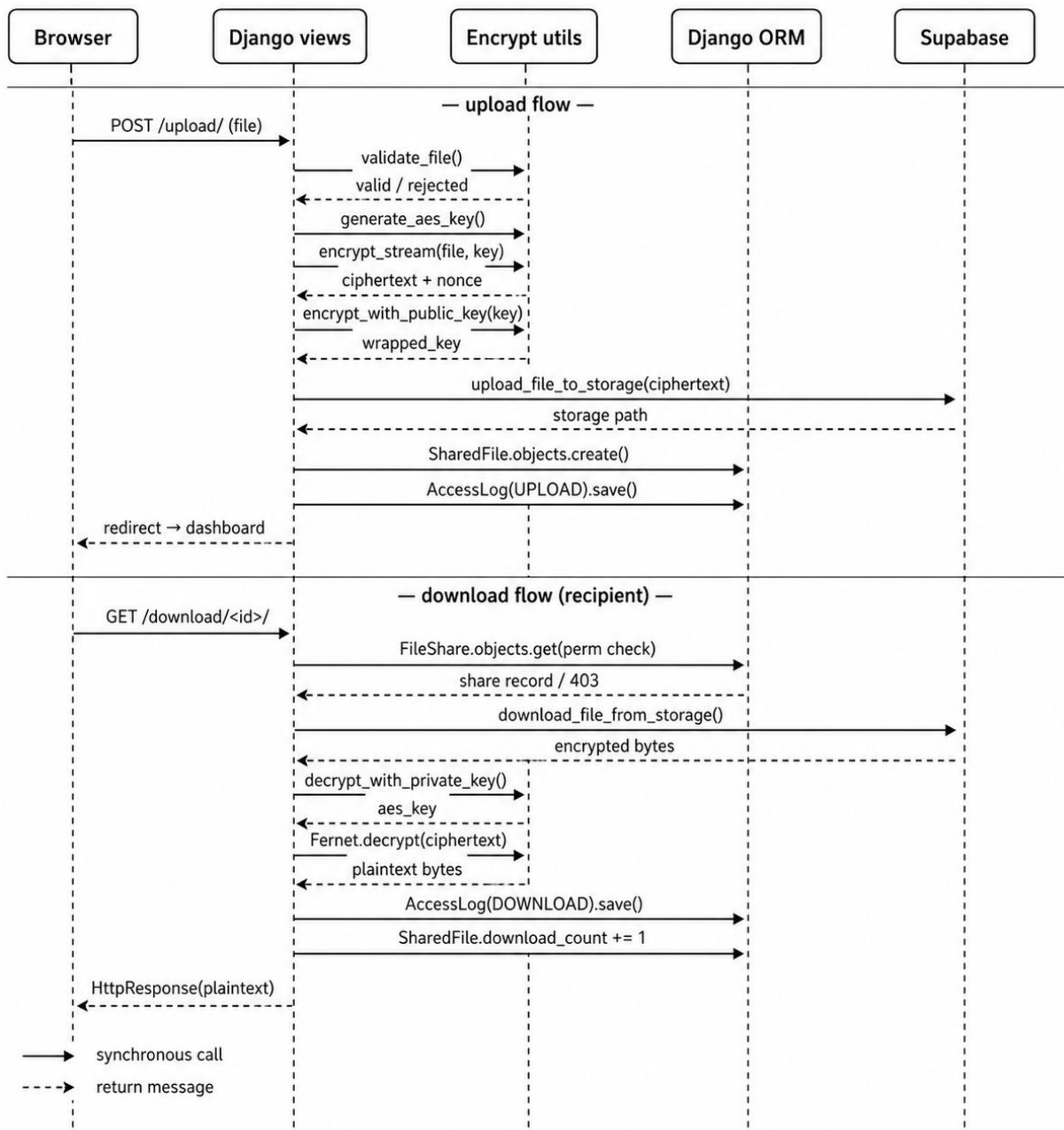


Figure 4.7: Sequence diagram for Secure File Sharing System

4.2.5 Timeline/Gantt Chart

The Gantt Chart shows the project timeline for developing the Cryptex secure file sharing system. It includes phases like analysis, design, frontend and backend development, testing, and deployment. Tasks are scheduled weekly for proper time management. It helps monitor progress, allocate resources, and ensure timely completion.

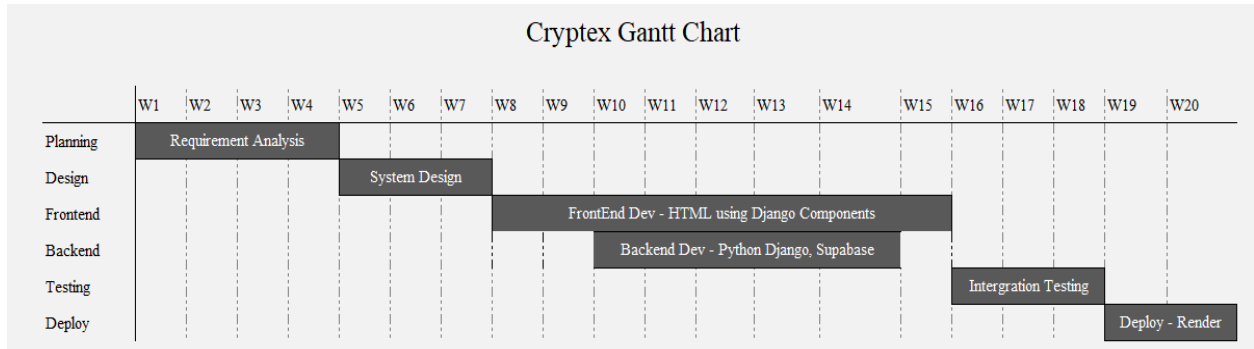


Figure 4.8: Timeline for Secure File Sharing System

4.1.4 Work Breakdown Structure (WBS) Chart

The Work Breakdown Structure (WBS) organizes the Cryptex project into phases like planning, analysis, development, testing, and deployment. It breaks each phase into tasks such as authentication, encryption, dashboard design, and security testing. This helps in planning, task management, progress tracking, and smooth project completion.

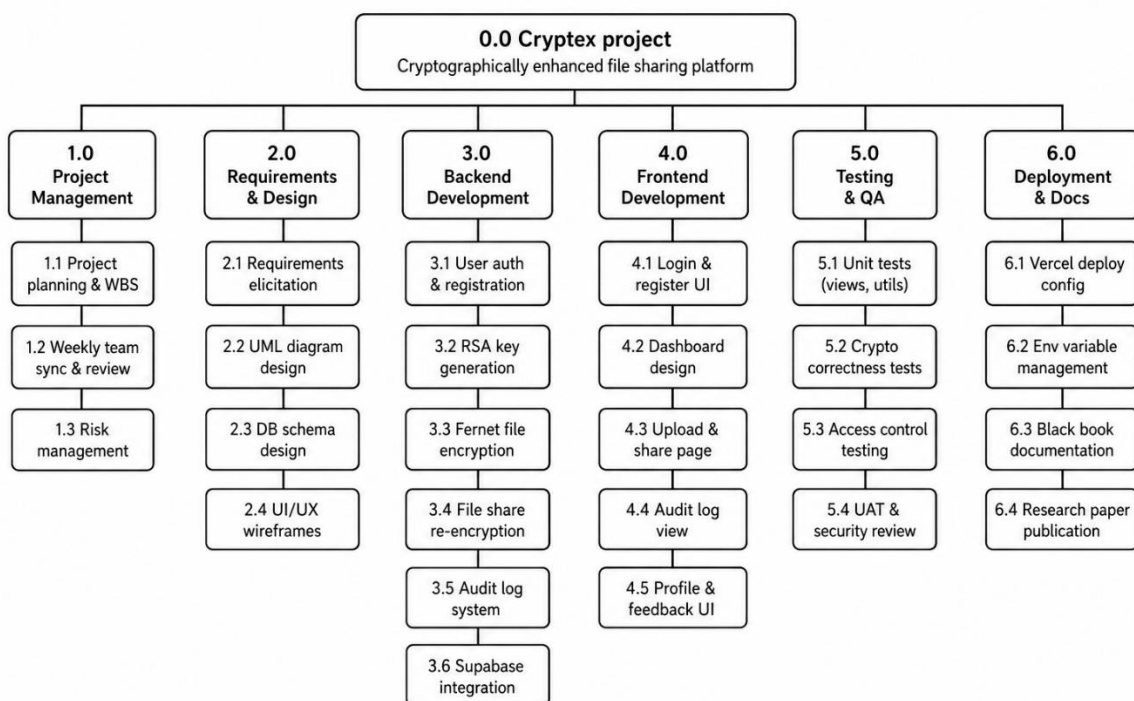


Figure 4.9: WBS Chart for Secure File Sharing System

CHAPTER 5

REPORT ON PRESENT INVESTIGATION

5.1 Proposed System

The proposed system is designed to provide a secure and efficient file-sharing platform using advanced cryptographic techniques. The main objective of the system is to ensure that files are protected from unauthorized access during both storage and transmission. The system follows a structured approach where users interact with a web-based interface to upload, download, and manage files securely.

In this system, when a user uploads a file, the file is not stored directly in its original form. Instead, it is first processed using encryption algorithms to convert it into a secure format. This ensures that even if the storage system is compromised, the data remains unreadable to unauthorized users. The encrypted files are then stored on the server along with necessary metadata such as file name, owner, and access permissions.

Access control mechanisms are used to ensure that only authorized users can access specific files. The system follows Role-Based Access Control (RBAC), where users are assigned roles such as admin or normal user. Based on these roles, permissions are given for actions like uploading, downloading, or sharing files. This helps prevent unauthorized access to sensitive data.

The system also includes an audit logging mechanism that records all user activities. These activities include login attempts, file uploads, downloads, and access requests. By maintaining these logs, the system ensures transparency and accountability. In case of any suspicious activity, administrators can review the logs and take necessary actions.

Additionally, the system ensures secure communication between the client and server using APIs. Data transmitted between the frontend and backend is protected using secure protocols, reducing the risk of data interception. The system is designed to maintain data integrity, ensuring that files are not modified or tampered with during storage or transfer.

The proposed system is scalable and can manage a growing number of users and files without reducing performance. It is designed with flexibility, allowing future upgrades such as multi-factor authentication, cloud integration, and stronger encryption methods. Overall, the methodology focuses on combining security, usability, and performance to build a reliable file-sharing system for real-world applications.

5.1.1 Architecture diagram of the System

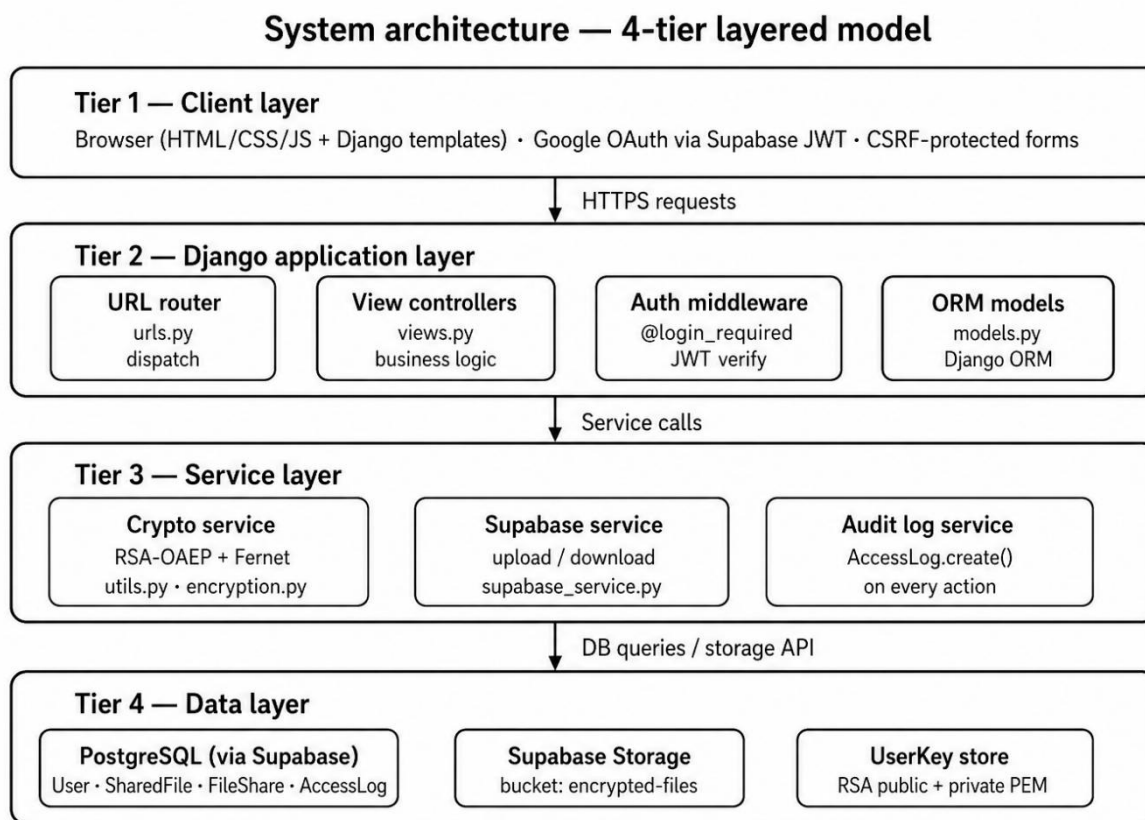


Figure 5.1: Architecture Diagram for Secure File Sharing System

5.2 Implementation

The implementation of the system is carried out using web technologies and security mechanisms to ensure efficient and secure file sharing. The following functionalities are implemented in the system:

1. The system allows users to register and log in securely using authentication mechanisms. Passwords are stored in encrypted form to prevent misuse in case of data breaches. Proper validation is applied to ensure secure login.
2. Files uploaded by users are encrypted before being stored on the server. This ensures that even if the storage is accessed by unauthorized users, the data remains protected and unreadable.
3. Role-Based Access Control (RBAC) is implemented to restrict file access to authorized users only. Different roles are defined, and permissions are assigned accordingly.
4. Users can securely download files, where decryption is performed only after proper authentication and authorization. This ensures that only valid users can access the original file.
5. Audit logs are maintained to record all user activities such as login, upload, and download. These logs help in monitoring system usage and detecting suspicious activities.

6. The system ensures secure communication between frontend and backend using APIs. Data transmission is protected using secure protocols, reducing the risk of interception.
7. File metadata and user details are stored securely in the database. Proper database security techniques are used to prevent unauthorized access and data leakage.
8. The system is designed to handle multiple users efficiently with proper security mechanisms. It ensures smooth performance even when multiple operations are performed simultaneously.
9. Error handling mechanisms are implemented to manage unexpected situations such as failed uploads or unauthorized access attempts. This improves system reliability.
10. The system also includes session management to ensure that user sessions are handled securely and expired after a certain period of inactivity.

5.2.1 Algorithm

A) Encryption Algorithm (AES — Encryption Process)

Step 1: Take the original file or message as input (Plaintext).

Step 2: Convert the plaintext data into binary format for processing.

Step 3: Generate a secret encryption key (128-bit, 192-bit, or 256-bit) using the key management module associated with the authenticated user's account.

Step 4: Apply key expansion to derive multiple round keys from the original secret key using the AES key schedule.

Step 5: Perform the initial round:

- Add Round Key — XOR the plaintext block with the first round key.

Step 6: Perform multiple intermediate rounds (9 rounds for 128-bit, 11 for 192-bit, 13 for 256-bit), where each round consists of:

- Substitute Bytes — replace each byte using the AES S-box substitution table.
- Shift Rows — cyclically shift the rows of the state matrix to the left.
- Mix Columns — multiply each column of the state matrix to provide diffusion.
- Add Round Key — XOR the state with the corresponding round key.

Step 7: Perform the final round (without Mix Columns step):

- Substitute Bytes — apply S-box substitution.
- Shift Rows — apply row shifting.
- Add Round Key — XOR with the final round key.

Step 8: Convert the resulting binary output into ciphertext format.

Step 9: Store the encrypted file (ciphertext) in the server database. The original plaintext file is discarded and never stored, ensuring data confidentiality at rest.

Output: Encrypted File (Ciphertext) securely stored in the database.

B) Decryption Algorithm (AES — Reverse Process)

Step 1: Retrieve the encrypted file (Ciphertext) from the server database upon an authorized access request.

Step 2: Verify the requesting user's identity and validate their access permissions against the access control list. If unauthorized, deny access and log the attempt in the audit trail.

Step 3: Retrieve the same secret key used during encryption from the secure key management module for the authorized user.

Step 4: Apply key expansion to regenerate all round keys from the secret key using the AES key schedule.

Step 5: Perform the initial reverse round:

- Add Round Key — XOR the ciphertext block with the last round key.

Step 6: Perform multiple reverse intermediate rounds, where each round consists of:

- Inverse Shift Rows — cyclically shift rows of the state matrix to the right.
- Inverse Substitute Bytes — reverse S-box substitution using the inverse S-box table.
- Add Round Key — XOR the state with the corresponding round key.
- Inverse Mix Columns — apply the inverse mixing transformation to each column.

Step 7: Perform the final reverse round (without Inverse Mix Columns):

- Inverse Shift Rows — apply inverse row shifting.
- Inverse Substitute Bytes — apply inverse S-box substitution.
- Add Round Key — XOR with the first round key.

Step 8: Convert the resulting binary data back into the original plaintext file format.

Step 9: Deliver the decrypted file to the authorized user for viewing or downloading.

Output: Original Decrypted File (Plaintext) delivered to the authorized user.

5.2.2 System Data

Table 5.2: User and File Metadata

Field Name	Description	Example
User_ID	Unique ID of user	U123
Username	Name of user	hardik123
Email	User email	hardikpawar000@gmail.com
Password_Hash	Encrypted password	SHA-256 hash
File_ID	Unique file identifier	F456
File_Name	Name of uploaded file	report.pdf
File_Size	Size of file	2MB
Upload_Date	Date of upload	01-04-2026
Owner_ID	User who uploaded file	U123
Encryption_Key	Key used for encryption	AES Key
Access_Permission	Who can access file	Private/Public

Table 5.3: Audit Log Data

Field Name	Description	Example
Log_ID	Unique log entry ID	L001
User_ID	User performing action	U123
Action	Activity performed	Upload / Download
File_ID	File involved	F456
Timestamp	Date & time of action	01-04-2026 10:30 AM
IP_Address	User IP	192.168.1.1
Status	Success / Failure	Success

5.2.3 GUI

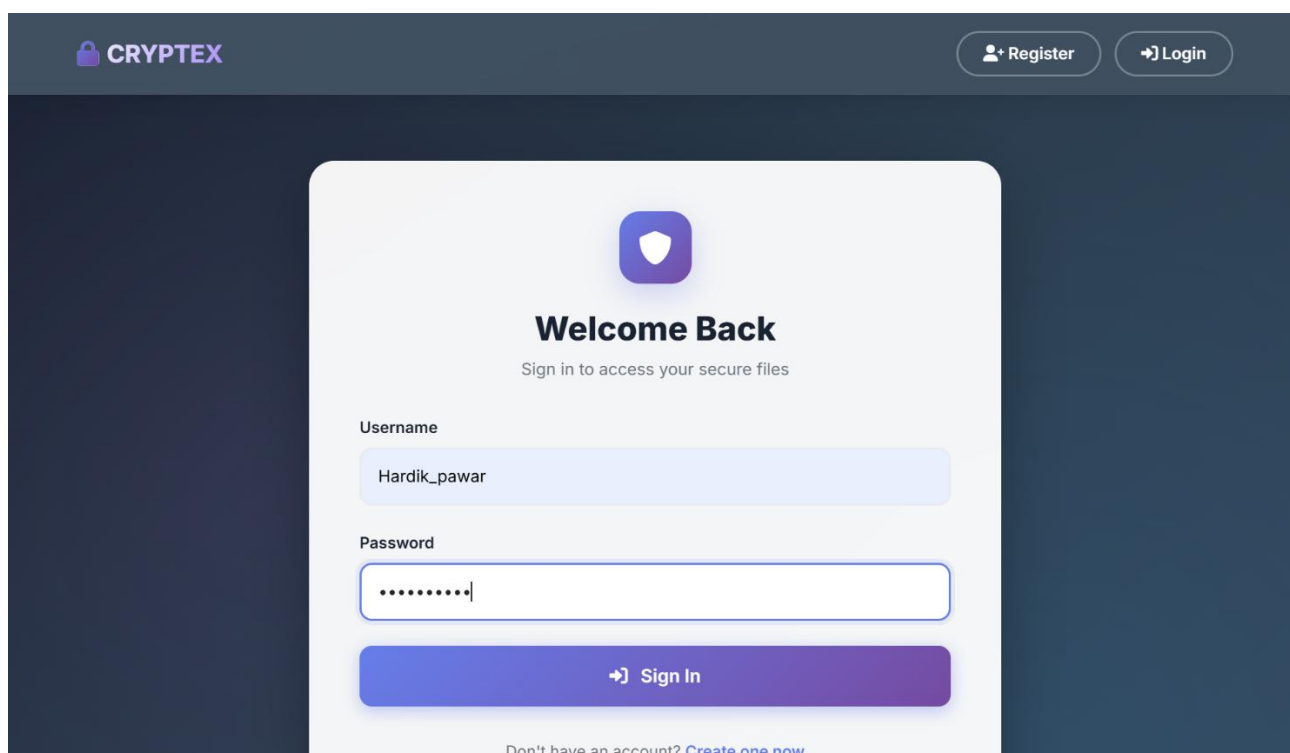


Figure 5.2.: User Login and Registration Page

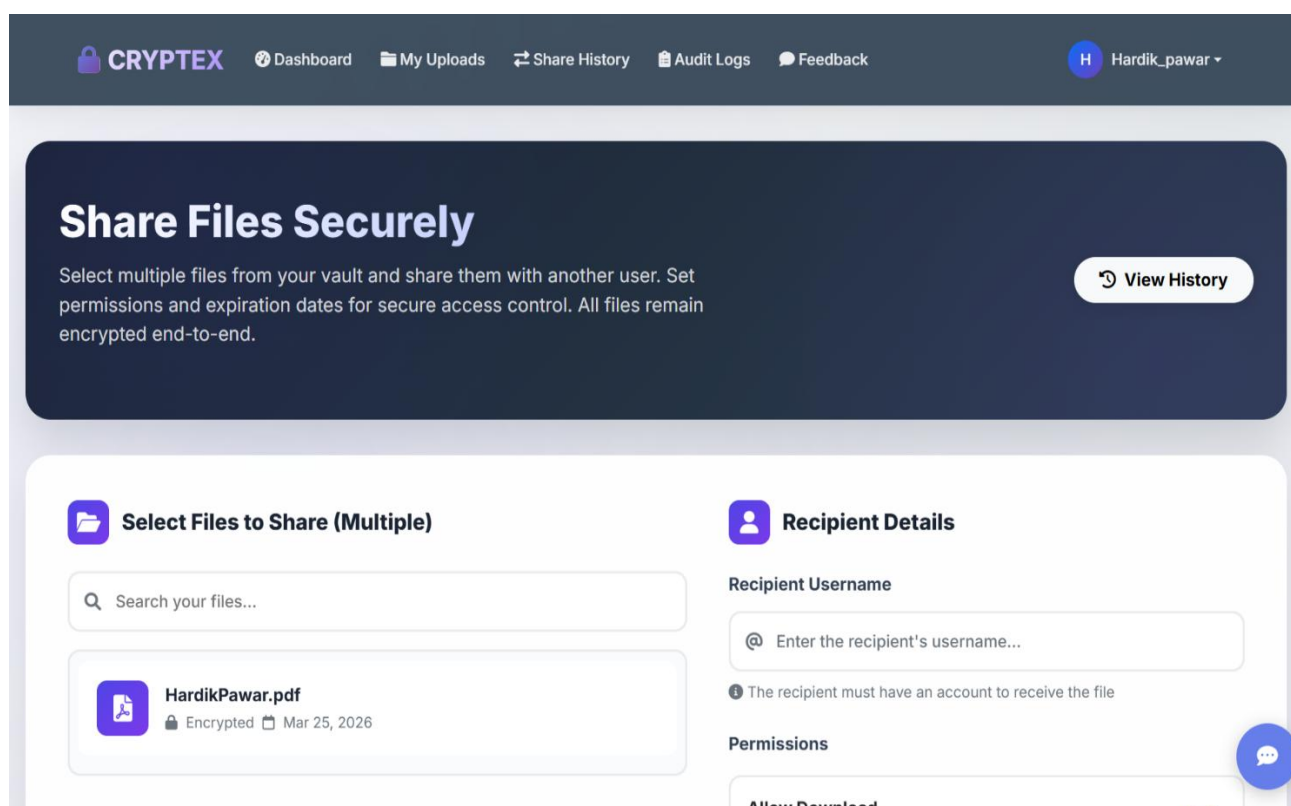


Figure 5.3: File Upload and Secure Sharing Interface

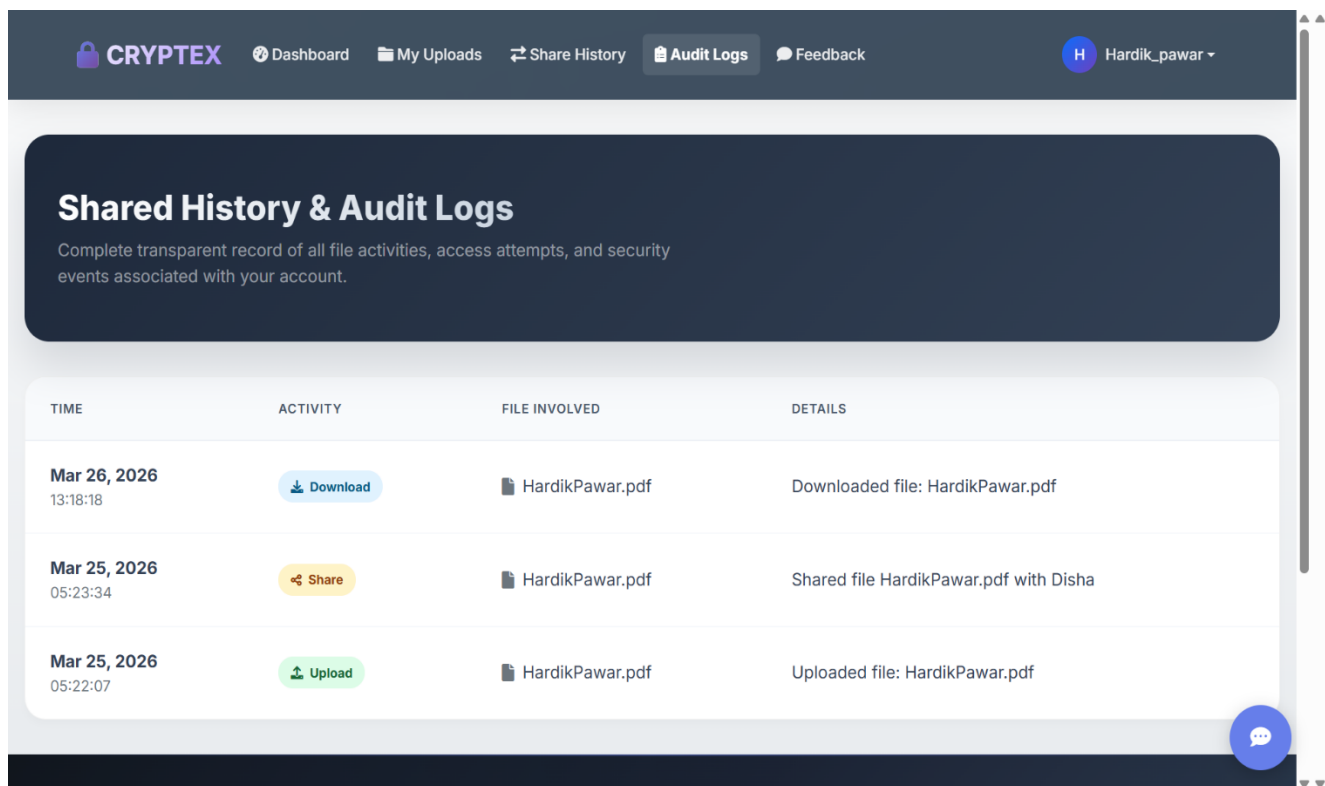


Figure 5.4: Audit Logs and Activity Monitoring Dashboard

CHAPTER 6

RESULTS AND DISCUSSION

After implementing the proposed secure file-sharing system, all the core functionalities were successfully tested and validated. The system demonstrates the ability to securely upload, store, and download files while maintaining data confidentiality, integrity, and controlled access. The results obtained from the system confirm that the integration of cryptographic techniques, access control, and audit logging mechanisms provides a reliable and secure environment for file sharing. The snapshots below represent the working of the system in real-time scenarios and help in understanding the practical implementation of the proposed solution.

6.1 User Registration and Login

- The system provides a secure user registration and login mechanism. New users can register by providing necessary details such as username, email, and password. During registration, input validation is applied to ensure that the data entered by the user is correct and secure. Passwords are stored in encrypted form in the database to prevent unauthorized access.
- Once registered, users can log in using their credentials. The system verifies the entered details with the stored records and grants access only if the credentials are valid. This ensures that only legitimate users can access the platform.

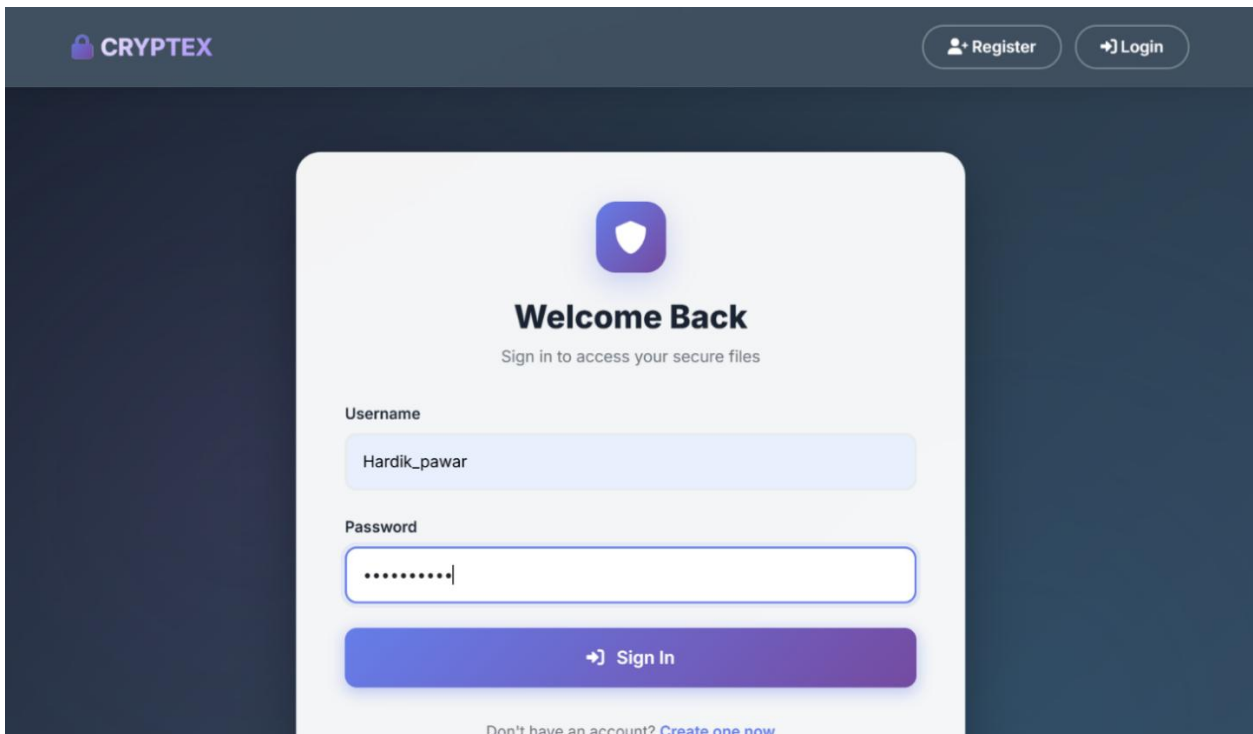
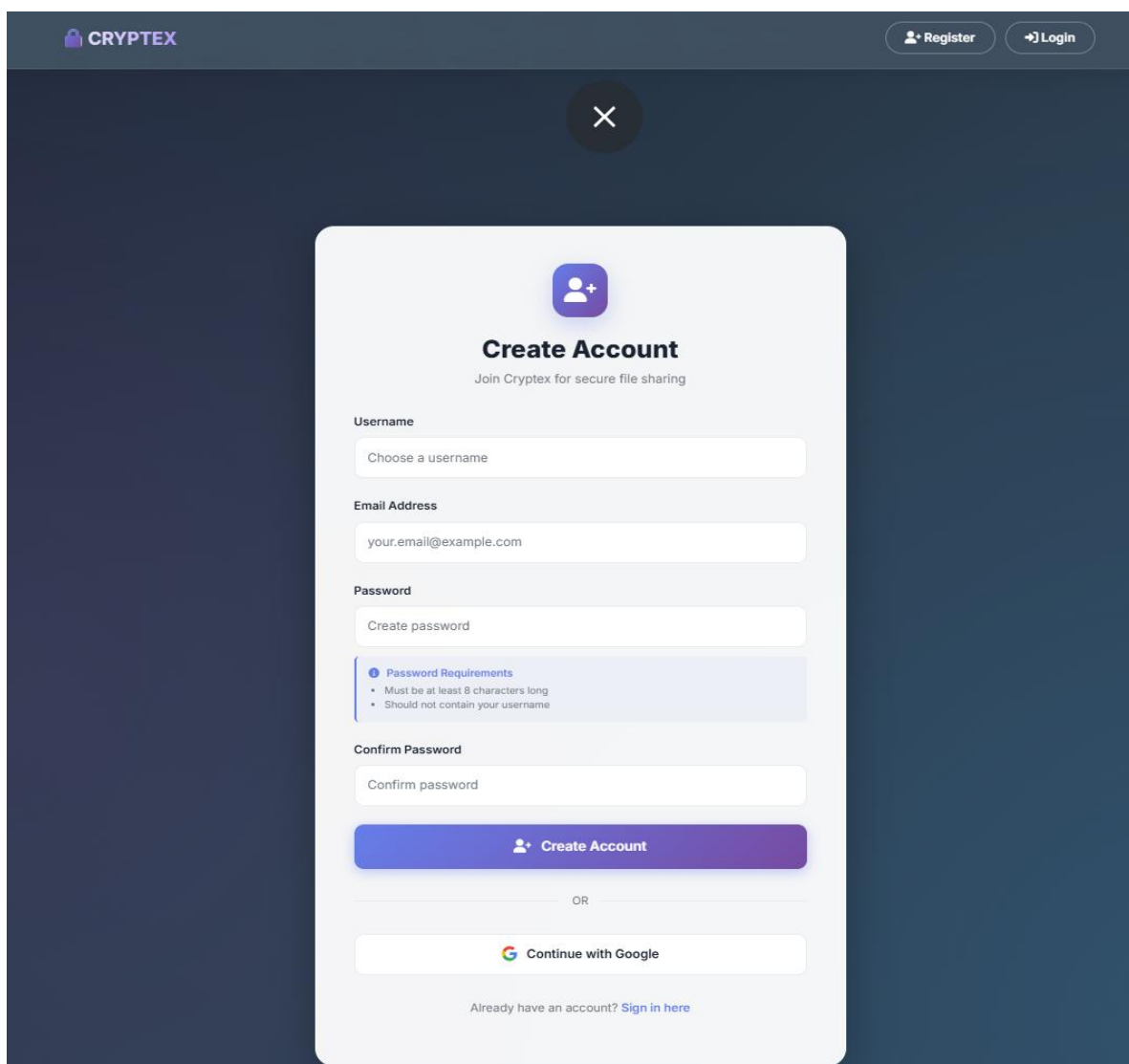


Figure 6.1: User Registration and Login

- The registration and login module is essential for maintaining system security. It prevents unauthorized users from accessing the system and ensures that all activities are performed by authenticated users. The implementation of secure authentication mechanisms improves overall system reliability and user trust.

6.2 User Authentication Process

- The system uses authentication techniques to verify the identity of users before granting access to system resources. This process ensures that only authorized individuals can perform actions such as uploading or downloading files.



The image shows a web application interface for CRYPTEX. At the top, there is a dark blue header with the CRYPTEX logo on the left and 'Register' and 'Login' buttons on the right. Below the header, a large dark blue modal window is centered, featuring a white 'X' close button in the top right corner. Inside the modal, the title 'Create Account' is displayed in bold, with a subtitle 'Join Cryptex for secure file sharing' below it. The form contains four input fields: 'Username' (placeholder: 'Choose a username'), 'Email Address' (placeholder: 'your.email@example.com'), 'Password' (placeholder: 'Create password'), and 'Confirm Password' (placeholder: 'Confirm password'). Below the password field, a 'Password Requirements' section lists two rules: 'Must be at least 8 characters long' and 'Should not contain your username'. A purple 'Create Account' button is positioned below the form fields. Below this button, the text 'OR' is centered, followed by a 'Continue with Google' button featuring the Google logo. At the bottom of the modal, a link reads 'Already have an account? Sign in here'.

Figure 6.2: User Authentication Process

6.3 Secure File Upload

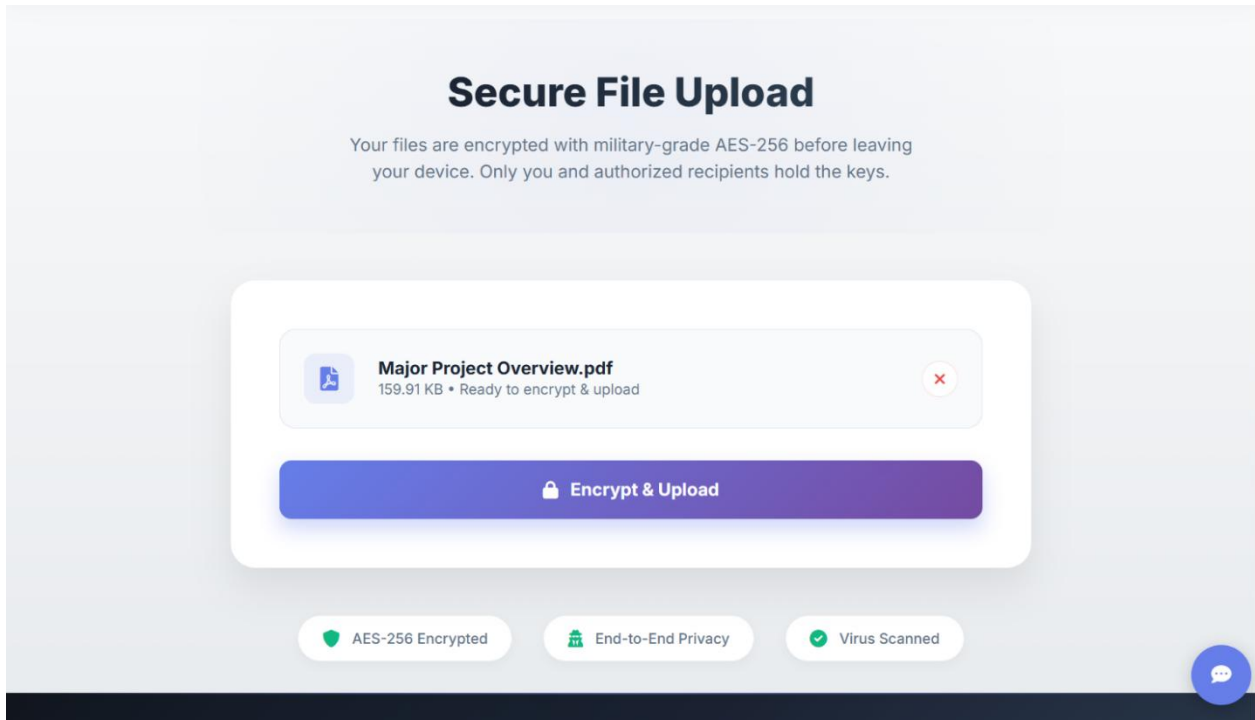


Figure 6.3: Secure File Upload

- After encryption, files can only be accessed by users with proper permissions. Role-Based Access Control ensures restricted access.
- The system allows secure downloading of files with decryption applied only for authorized users.

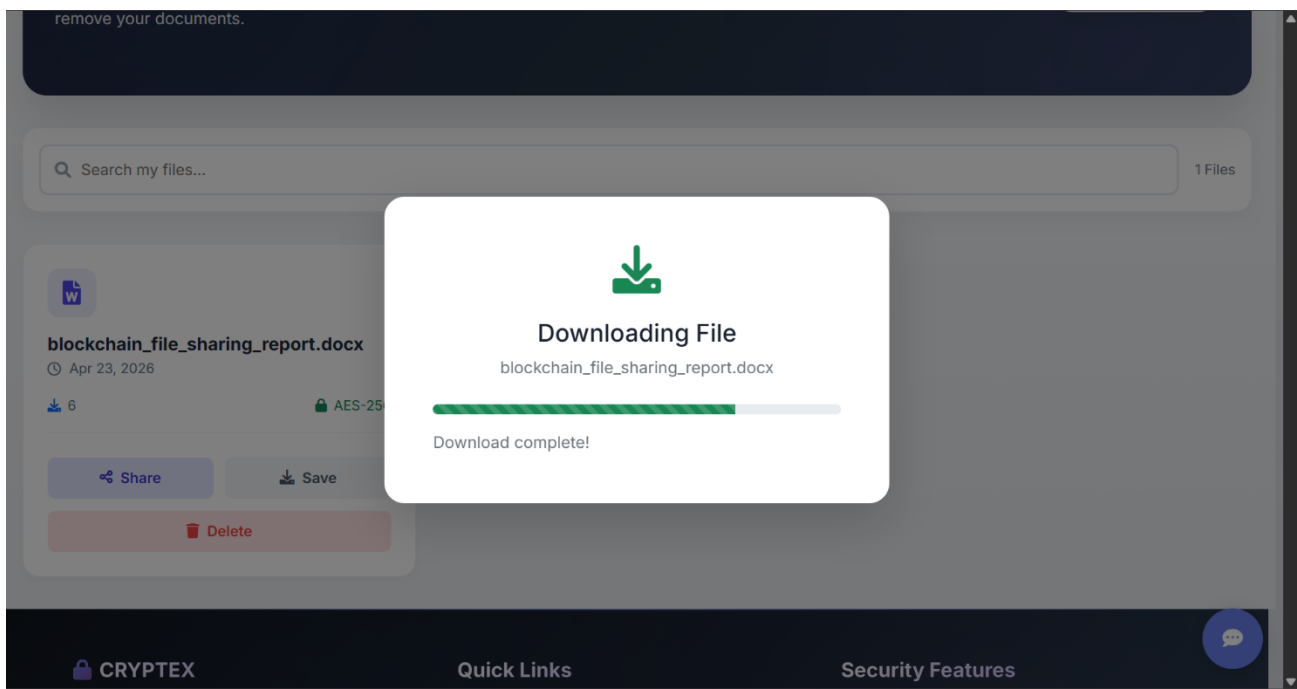


Figure 6.4: Secure File Download

- Audit logs are generated to track all user activities, such as login, file upload, and download.

TIME	ACTIVITY	FILE INVOLVED	DETAILS
Mar 26, 2026 13:18:18	Download	HardikPawar.pdf	Downloaded file: HardikPawar.pdf
Mar 25, 2026 05:23:34	Share	HardikPawar.pdf	Shared file HardikPawar.pdf with Disha
Mar 25, 2026 05:22:07	Upload	HardikPawar.pdf	Uploaded file: HardikPawar.pdf

Figure 6.5: Audit Logs and Activity Monitoring

- The system ensures data confidentiality and prevents unauthorized access through encryption and access control mechanisms. It also provides transparency by maintaining activity records.

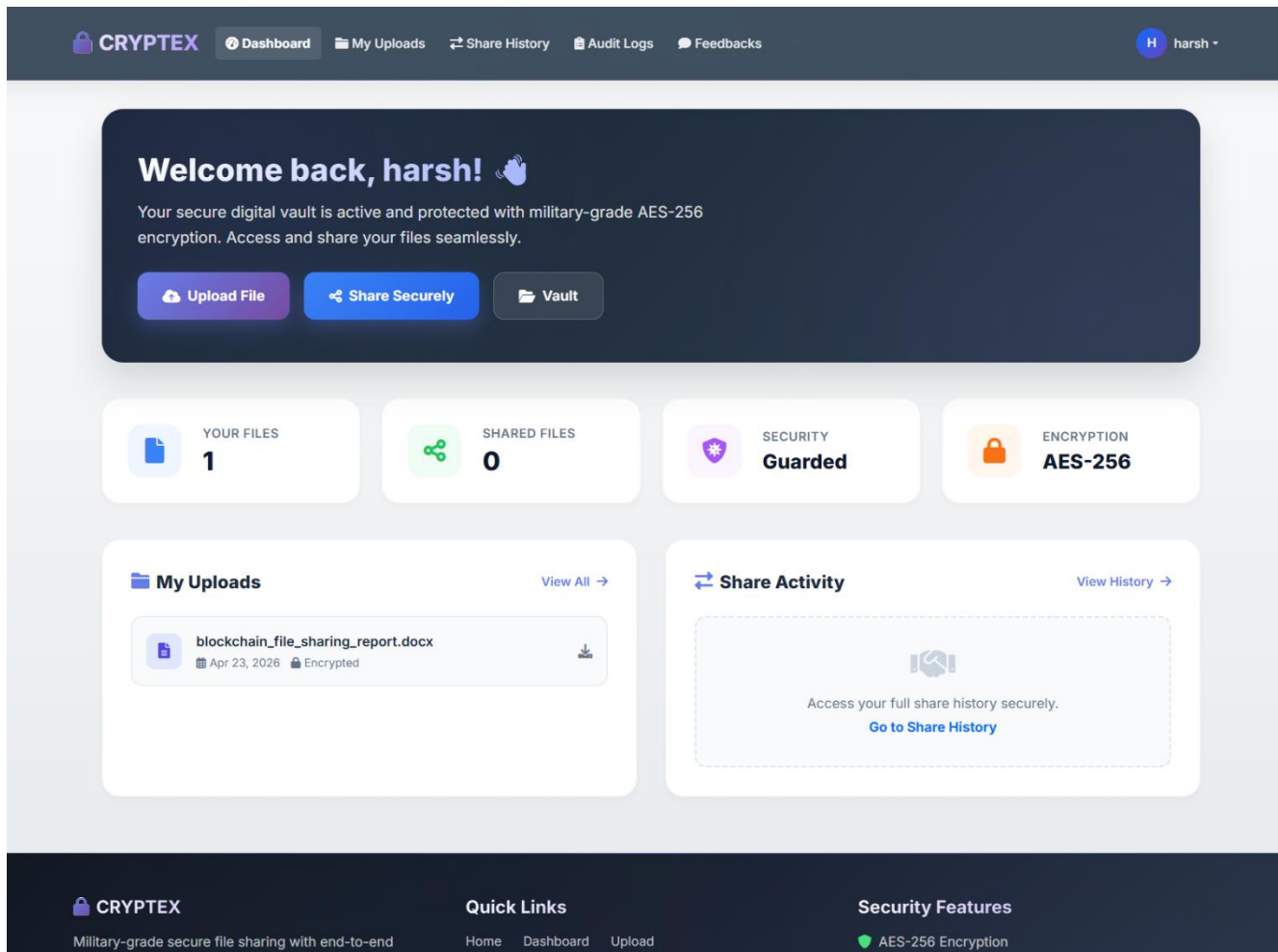


Figure 6.6: System Security and Access Control

- The proposed system focuses on secure file sharing by implementing encryption, controlled access, and monitoring mechanisms. It can be further enhanced with additional security features to improve performance and scalability.

CHAPTER 7

CONCLUSION

Secure file-sharing systems play a vital role in protecting sensitive data in today's digital world. This project focused on developing a Cryptographically Enhanced Web Platform for Controlled and Auditable File Sharing that ensures strong data security using cryptographic techniques such as encryption and decryption. By encrypting files before storage and transfer, the system helps maintain confidentiality and prevents unauthorized users from accessing sensitive information.

The proposed system addresses many limitations of traditional file-sharing platforms by providing secure data transmission, controlled access, and proper monitoring mechanisms. Only authorized users with valid permissions can access shared files, which reduces the chances of data leakage, misuse, and cyber threats. The implementation of secure authentication further strengthens the protection of user accounts and system resources.

Role-Based Access Control (RBAC) plays an important role in managing permissions within the system. Different privileges are assigned to users and administrators based on their responsibilities. This ensures that critical operations such as user management, file monitoring, and access control remain restricted to authorized personnel only.

The inclusion of audit logging improves transparency and accountability. All important activities such as registration, login attempts, file uploads, downloads, sharing actions, and access requests are recorded in logs. These logs help administrators monitor the system, detect suspicious behavior, and take necessary action when required.

The system also provides a user-friendly web interface that allows users to upload, share, and download files efficiently without compromising security. Integration with cloud storage enables reliable file management and improves system scalability. The platform is capable of handling multiple users while maintaining performance and security.

Overall, the proposed system improves trust, reliability, and efficiency in digital file-sharing processes. It can be effectively used in organizations, educational institutions, healthcare sectors, and other environments where secure data exchange is essential. The project successfully demonstrates how cryptography, access control, and auditing can be combined to create a practical and dependable secure file-sharing solution.

FUTURE SCOPE

- Multi-Factor Authentication (MFA) can be added to improve login security through OTP, email verification, or biometric authentication.
- Blockchain-based audit logs can be implemented to make activity records tamper-proof and more transparent.
- AI-based threat detection can be integrated to identify suspicious login attempts, abnormal downloads, or malicious behavior automatically.
- Large-scale cloud integration with multiple storage providers can improve backup, availability, and performance.
- Mobile application support for Android and iOS can be developed for secure file access on smartphones.
- Advanced sharing controls such as time-limited links, one-time downloads, and watermark protection can be introduced.
- End-to-End Encryption can be enhanced so that only sender and receiver can decrypt files, even the server cannot access them.
- Real-time notifications and alerts can be added for file access, sharing requests, and security incidents.
- Support for collaboration features such as secure team sharing, comments, and version control can be implemented.
- The system can be expanded for enterprise-level deployment with high scalability, load balancing, and centralized management.

References

- [1] N. Vurukonda and B. T. Rao, "A Study on Data Storage Security Issues in Cloud Computing," *Procedia Computer Science*, vol. 92, pp. 128–135, 2016, doi: 10.1016/j.procs.2016.07.335.
- [2] B. S. Rawal and S. S. Vivek, "Secure Cloud Storage and File Sharing," 2017 IEEE International Conference on Smart Cloud (SmartCloud), New York, NY, USA, 2017, pp. 78-83, doi: 10.1109/SmartCloud.2017.19.
- [3] O. A. Khashan, "Secure Outsourcing and Sharing of Cloud Data Using a User-Side Encrypted File System," *IEEE Access*, vol. 8, pp. 210855–210867, 2020, doi: 10.1109/ACCESS.2020.3039163.
- [4] K. Negi, R. Shrestha, T. L. Borges, S. Sahana and S. Das, "A Hybrid Cryptographic Approach for Secure Cloud-Based File Storage," 2023 IEEE IAS Global Conference on Emerging Technologies (GlobConET), London, United Kingdom, 2023, pp. 1–7, doi: 10.1109/GlobConET56651.2023.10150148.
- [5] Z. Qin, H. Xiong, S. Wu and J. Batamuliza, "A Survey of Proxy Re-Encryption for Secure Data Sharing in Cloud Computing," *IEEE Transactions on Services Computing*, doi: 10.1109/TSC.2016.2551238.
- [6] I. Gupta, A. K. Singh, C.-N. Lee and R. Buyya, "Secure Data Storage and Sharing Techniques for Data Protection in Cloud Environments: A Systematic Review, Analysis, and Future Directions," *IEEE Access*, vol. 10, pp. 71247–71277, 2022, doi: 10.1109/ACCESS.2022.3188110.
- [7] P. Yang, N. Xiong and J. Ren, "Data Security and Privacy Protection for Cloud Storage: A Survey," *IEEE Access*, vol. 8, pp. 131723–131740, 2020, doi: 10.1109/ACCESS.2020.3009876.
- [8] R. Pitchai, S. Jayashri and J. Raja, "Searchable Encrypted Data File Sharing Method Using Public Cloud Service for Secure Storage in Cloud Computing," *Wireless Personal Communications*, vol. 90, pp. 947–960, 2016, doi: 10.1007/s11277-016-3273-1.
- [9] M. Ali et al., "SeDaSC: Secure Data Sharing in Clouds," *IEEE Systems Journal*, vol. 11, no. 2, pp. 395–404, Jun. 2017, doi: 10.1109/JSYST.2014.2379646.
- [10] S. Xu, G. Yang, Y. Mu and R. H. Deng, "Secure Fine-Grained Access Control and Data Sharing for Dynamic Groups in the Cloud," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 8, pp. 2101–2113, Aug. 2018, doi: 10.1109/TIFS.2018.2810065.

- [11] R. Li, C. Shen, H. He, X. Gu, Z. Xu and C.-Z. Xu, "A Lightweight Secure Data Sharing Scheme for Mobile Cloud Computing," *IEEE Transactions on Cloud Computing*, vol. 6, no. 2, pp. 344–357, Apr.–Jun. 2018, doi: 10.1109/TCC.2017.2649685.
- [12] W. Li, K. Xue, Y. Xue and J. Hong, "TMACS: A Robust and Verifiable Threshold Multi-Authority Access Control System in Public Cloud Storage," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 5, pp. 1484–1496, May 2016, doi: 10.1109/TPDS.2015.2448095.
- [13] Y.-W. Kao, K.-Y. Huang, H.-Z. Gu and S.-M. Yuan, "uCloud: A User-Centric Key Management Scheme for Cloud Data Protection," *IET Information Security*, vol. 7, pp. 144–154, 2013, doi: 10.1049/iet-ifs.2012.0198
- [14] G. Viswanath and P. V. Krishna, "Hybrid Encryption Framework for Securing Big Data Storage in Multi-Cloud Environment," *Evolutionary Intelligence*, vol. 14, pp. 691–698, 2021, doi: 10.1007/s12065-020-00404-w.
- [15] V. Verma, P. Kumar, R. K. Verma and S. Priya, "A Novel Approach for Security in Cloud Data Storage Using AES-DES-RSA Hybrid Cryptography," *2021 Emerging Trends in Industry 4.0 (ETI 4.0)*, Raigarh, India, 2021, pp. 1–6, doi: 10.1109/ETI4.051663.2021.9619274.
- [16] V. S. Mahalle and A. K. Shahade, "Enhancing the Data Security in Cloud by Implementing Hybrid (RSA & AES) Encryption Algorithm," *2014 International Conference on Power, Automation and Communication (INPAC)*, Amravati, India, 2014, pp. 146–149, doi: 10.1109/INPAC.2014.6981152.

APPENDIX

TECHNOLOGIES USED

A. Django

Django is a high-level, open-source Python web framework used for developing secure and scalable web applications. It follows the Model-View-Template (MVT) architecture and provides built-in features such as authentication, ORM, admin panel, and security protection. In Cryptex, Django is used as the core backend framework to manage user authentication, file operations, access permissions, audit logs, and API handling. It simplifies rapid development while maintaining strong security standards.

B. Python

Python is a powerful, general-purpose programming language known for its simplicity, readability, and extensive library support. It is widely used in web development, cybersecurity, automation, and data processing. In Cryptex, Python is used for implementing backend logic, encryption/decryption processes, secure authentication workflows, and server-side data handling. Its cryptographic and web libraries make it suitable for building secure systems.

C. HTML, CSS and JavaScript

HTML, CSS, and JavaScript are the core technologies used for creating modern web interfaces. HTML provides page structure, CSS handles styling and layout, while JavaScript adds dynamic and interactive behavior. In Cryptex, these technologies are used to design responsive user interfaces such as login pages, dashboards, file upload forms, file management screens, and admin panels. They ensure a smooth and user-friendly browsing experience.

D. Supabase

Supabase is an open-source Backend-as-a-Service (BaaS) platform that provides database, authentication, storage, and real-time features. It is built on PostgreSQL and offers scalable cloud infrastructure. In Cryptex, Supabase is used for secure cloud file storage, metadata management, and maintaining user-related data. It enables efficient storage and retrieval of encrypted files while ensuring reliability and performance.

E. PostgreSQL

PostgreSQL is a powerful open-source relational database management system known for reliability, scalability, and advanced SQL features. It is widely used for enterprise applications requiring secure data storage. In Cryptex, PostgreSQL (through Supabase) is used to store user records, file metadata, sharing permissions, audit logs, and system activity data. It ensures consistency and structured management of critical information.

F. Cryptography Libraries

Cryptography libraries provide tools for implementing secure encryption, decryption, hashing, and key generation techniques. They are essential for protecting sensitive data against unauthorized access. In Cryptex, Python cryptographic libraries are used for file encryption, secure key handling, password hashing, and decryption during authorized downloads. These libraries form the security backbone of the platform.

G. Google OAuth Authentication

Google OAuth is a secure authentication protocol that allows users to log in using their Google accounts without creating separate passwords. It improves convenience and reduces password management issues. In Cryptex, Google OAuth is integrated through Django-Allauth to provide secure social login functionality for users.

H. PBKDF2 Password Hashing

PBKDF2 (Password-Based Key Derivation Function 2) is a secure password hashing algorithm used to protect stored passwords through salting and multiple iterations. It is Django's default password hashing mechanism. In Cryptex, PBKDF2 is used to securely hash user passwords before storing them in the database, protecting accounts from brute-force attacks and password leaks.

I. Git and GitHub

Git is a distributed version control system used for tracking source code changes, while GitHub is a cloud platform for hosting repositories and collaboration. They help teams manage code efficiently and maintain project versions. In Cryptex, Git and GitHub are used for source code management, backup, collaboration, and deployment workflow tracking.

J. WhiteNoise

WhiteNoise is a lightweight Python package used for serving static files efficiently in production environments. It simplifies deployment by allowing Django applications to serve CSS, JavaScript, and image files directly. In Cryptex, WhiteNoise is used for managing static assets and improving deployment performance.

K. Gunicorn

Gunicorn is a Python WSGI HTTP server used for deploying web applications in production. It is lightweight, reliable, and commonly used with Django projects. In Cryptex, Gunicorn is used to run the Django application on the server and handle incoming web requests efficiently.

L. Vercel

Vercel is a modern cloud deployment platform used for hosting web applications with fast global delivery and automatic CI/CD integration. It supports frontend and backend deployments with scalability. In Cryptex, Vercel is used for deploying the project interface and making the secure file-sharing platform accessible online with reliable performance.

PUBLICATIONS

- [1] Hardik Pawar, Harsh Nimbare, Aryan Kasekar, Nandini Baisane and Pooja Gharat. “A Cryptographically Enhanced Web Platform for Controlled and Auditable File Sharing” *In Journal of Engineering and Technology for Industrial Application (JETIA)* pp. xxx-xxx. ITEGAM-JETIA, DOI: <https://doi.org/10.5935/jetia.v12i58.3167>

ACKNOWLEDGMENT

We would like to express our sincere gratitude to our Principal, **Dr. Kamal Shah**, Principal of St. John College of Engineering and Management, for providing us with the facilities, support, and encouragement to complete our major project successfully. We also extend our heartfelt thanks to our Head of Department, **Dr. Arun Saxena**, HOD, Department of Information Technology, for his valuable guidance and continuous motivation throughout the project. We are deeply grateful to our Guide, **Mrs. Pooja Gharat**, Assistant Professor, Department of Information Technology, for his constant support, valuable feedback, and insightful suggestions that helped us accomplish this work effectively. We also wish to thank our colleagues and friends for their cooperation and teamwork during the development of this project. Finally, we owe our deepest gratitude to our parents for their constant encouragement, moral support, and blessings, which have been our inspiration throughout this academic journey.

Hardik Tushar Pawar [EU1214036]

Harsh Sadanand Nimbare [EU1224041]

Aryan Gajanan Kasekar [EU1224040]

Nandini Satyavijay Baisane [EU1224022]